



AJAR: Aggregations and Joins over Annotated Relations

Manas Joglekar
Stanford University
manasrj@stanford.edu

Rohan Puttagunta
Stanford University
rohanp@stanford.edu

Christopher Ré
Stanford University
chrismre@cs.stanford.edu

ABSTRACT

We study a class of aggregate-join queries with multiple aggregation operators evaluated over annotated relations. We show that straightforward extensions of standard multiway join algorithms and generalized hypertree decompositions (GHDs) provide best-known runtime guarantees. In contrast, prior work uses bespoke algorithms and data structures and does not match these guarantees. We extend the standard techniques by providing a complete characterization of (1) the set of orderings equivalent to a given ordering and (2) the set of GHDs valid with respect to the given ordering, i.e., GHDs that correctly answer a given aggregate-join query when provided to (simple variants of) standard join algorithms. We show by example that previous approaches are incomplete. The key technical consequence of our characterizations is a decomposition of a valid GHD into a set of (smaller) *unconstrained* GHDs, i.e., into a set of GHDs of sub-queries without aggregations. Since this decomposition is comprised of unconstrained GHDs, we are able to connect to the wide literature on GHDs for join query processing, thereby obtaining improved runtime bounds, MapReduce variants, and an efficient method to find approximately optimal GHDs.

1. INTRODUCTION

Generalized hypertree decompositions (GHDs), introduced by Gottlob et al. [9, 10] and further developed by Grohe and Marx [12], provide a means for performing early projection in join processing, which can result in dramatically faster runtimes. In this work, we extend GHDs to handle queries that include aggregations, which allows us to capture both SQL-aggregate processing and message passing problems. Motivated by our own database engine based on GHDs [1, 23, 27], we seek to more deeply understand the space of optimization for aggregate-join queries.

We build upon work by Green, Karvounarakis, and Tanen [11] on annotated relations to define our notion of aggregation. These annotations provide a general definition of aggregation, allowing us to represent a wide-ranging set

R			S			$R \bowtie S$			
A	B	Z	B	C	Z	A	B	C	Z
1	3	3	1	1	4	1	3	3	18
1	2	1	3	3	6	1	1	1	8
1	1	2							

$\sum_C \sum_B R \bowtie S$	
A	Z
1	26

Figure 1: Illustrating the computation of Example 1

of problems as aggregate-join queries. Our queries, which we call AJAR (Aggregations and Joins over Annotated Relations) queries, contain semiring quantifiers that “sum over” or “marginalize out” values. We formally define AJAR queries in Section 3, but they are easy to illustrate by example:

EXAMPLE 1. Consider two relations with attributes $\{A, B\}$ and $\{B, C\}$ such that each tuple is annotated with some integer; we call these relations $\mathbb{Z}$ -relations. Consider the query:

$$\sum_C \sum_B R(A, B) \bowtie S(B, C)$$

Our output will then be a $\mathbb{Z}$ -relation with attribute set $\{A\}$. The tuples in the relation will simply be the tuples of $\pi_A R(A, B) \bowtie S(B, C)$; we note that our convention of aggregation is opposite to that of projection in relational algebra. Each value a of attribute A in R is associated with a set X_a of tuples (b, c) composed of the values that appear with a in $R(A, B) \bowtie S(B, C)$; $X_a = \pi_{B, C} \sigma_{A=a} R(A, B) \bowtie S(B, C)$. Furthermore let z_R and z_S be the functions that return annotations of a given tuple for relations R and S , respectively. Then the annotation associated with a in our output will simply be

$$\sum_{(b, c) \in X_a} z_R(a, b) \times z_S(b, c).$$

AJAR queries capture both classical SQL-style project-join queries and newer data processing problems like probabilistic inference via message passing on graphical models [15]. In fact, Aji and McEliece proposed the “Marginalize a Product Function” (MPF) problem [4], which is a special case of an AJAR query, and showed how the problem and its solution capture a number of classic problems and algorithms, including fast Hadamard transforms, Viterbi’s algorithm, forward-backward algorithm, FFT, and probabilistic inference in Bayesian networks. These algorithms are fundamental to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

PODS’16, June 26–July 01, 2016, San Francisco, CA, USA

© 2016 ACM. ISBN 978-1-4503-4191-2/16/06...\$15.00

DOI: <http://dx.doi.org/10.1145/2902251.2902293>

various fields; for example the forward-backward algorithm over conditional random fields forms the basis for state of the art solutions to named entity recognition, part of speech tagging, noun phrase segmentation, and other problems in NLP [26]. We are motivated by the wide applicability of queries over annotated relations; annotated relations may provide a framework for combining classical query processing, linear algebra, and statistical inference in a single data processing system.

We consider a generalization of MPF with multiple aggregation operators. We represent an aggregate-join query as a join and an aggregation ordering, which specifies both the ordering and the aggregation of each attribute. Our class of queries directly follows from the work of Abo Khamis, Ngo, and Rudra [16], who investigated the “Functional Aggregate Query” (FAQ) problem. In addition to MPF, FAQ is a generalization of Chen and Dalmau’s QCQ problem [7], in which the only aggregates are logical quantifiers (AND and OR).

The key technical challenge in these problems is characterizing the permissible aggregation orders to answer the query. A query consists of body Q and a string of attribute-operator pairs α , which we call an aggregation ordering. The challenge arises because different aggregation operators (e.g., $\sum$ and $\max$) cannot freely commute. We show that attribute-operator pairs can commute for only two reasons: (1) their operators commute or (2) their attributes are “independent” in the query, e.g., in query $\max_B \max_A \sum_C R(A, B), S(B, C)$, the aggregations involving A and C can commute – even though $\max$ and $\sum$ do not commute as operators, the query body renders them *independent* given B . These two conditions for commuting generate equivalence classes among aggregation orderings.

Given the ordering α , we wish to identify which orderings are equivalent to α and which GHDs are compatible with an equivalent ordering. Chen and Dalmau give a complete characterization of which variable orders are equivalent for QCQ via a procedure. In the same spirit, we provide such a characterization for both equivalent orderings and compatible GHDs. We describe these characterizations in turn.

- *A Complete Characterization of Equivalent Orders:* Given an order α and two attribute-operator pairs $x, y \in \alpha$, we describe a set of constraints of the form “*in any order, x must appear after y.*” Our constraints are sound and complete, i.e., a string β satisfies these constraints if and only if it is equivalent to α . In contrast, previous approaches have incomplete characterizations.
- *A Complete Characterization of GHDs Compatible with any Equivalent Order.* Given an order α and a query hypergraph Q , we call a GHD ‘valid’ if it is compatible with any ordering equivalent to α . We give a succinct characterization for all valid GHDs. We then describe a decomposition of the query (Q, α) into a series of *characteristic hypergraphs* (without attached aggregation orderings). GHDs for these hypergraphs can be combined into a valid GHD for the original query. We show that for any “node-monotone”¹ width function, there is a GHD with optimal width w that can be constructed with this decomposition. Treewidth, frac-

tional hypertreewidth, and submodular width are all node-monotone.

Conceptually, the latter result is important for tying our work to existing GHD literature; the result reduces our problem to constructing standard GHDs. Pragmatically, it lets us apply existing GHD results to our characteristic hypergraphs and obtain the following results for free:

- Based on Grohe and Marx [12], we are able to describe our runtime in terms of classical metrics like fractional hypertreewidth. In turn, we can use tighter notions of width to upper bound the runtime like Marx’s submodular width [18] or Joglekar’s efficiently computable variant [14].
- Based on Afrati et al. [3], who bound the communication costs of join processing in terms of a “width” parameter for GHDs, we can develop efficient MapReduce algorithms for solving AJAR queries.
- Based on Marx’s approximation [17] for GHDs, we can find approximately optimal GHDs for the popular fractional hypertreewidth measure in polynomial time.

We get the above results essentially for free from forging this connection to GHDs. We view this simple link as a strength of our approach.

Finally, we discuss an extension to handle “product aggregations” that allows us to aggregate away an attribute *before* we join the relations containing the attribute when the aggregation operator is the multiplication operator of the semiring. FAQ was the first to observe that this special case can improve certain types of logical queries. This opens up a new space of equivalent orderings and valid GHDs; mirroring the above results, we give a simple test and a complete characterization of the valid GHDs for queries that include this aggregation. This extension allows us to recover the results on QCQ presented by Chen and Dalmau. Moreover, we obtain similar improvements in runtime as previous work.

Outline. We discuss related work in Section 2. In Section 3, we introduce notation and algorithms that are relevant to our work before defining the AJAR problem. We also present our algorithm for answering AJAR queries. Section 4 provides a succinct characterization of all orderings that are equivalent to a given ordering. Section 5 discusses how to connect our work with recent research on GHDs, explaining how to construct valid optimal query plans and how to further improve and parallelize our results. In Section 6, we discuss how to incorporate product aggregations.

2. RELATED WORK

Join Algorithms. The Yannakakis algorithm, introduced in 1981, guarantees a runtime of $O(\text{IN} + \text{OUT})$ for α -acyclic join queries [29]. Modern multiway algorithms can process any join query and have worst-case optimal runtime. In particular, Atserias, Grohe, and Marx [5] derived a tight bound on the worst-case size of a join query given the input size and structure. Ngo et al. [20] presented the first algorithm to achieve this runtime bound, i.e., the first worst-case optimal algorithm. Soon after, Veldhuizen presented Leapfrog Triejoin, a very simple worst-case optimal algorithm that had been implemented in LogicBlox’s commercial database system [28]. Ngo et al. [21] later presented the simplified and

¹Informally, a measure is *node monotone* if adding more nodes to a graph does not reduce the measure, but additional edges may reduce the measure (see Definition 31).

unified algorithm GenericJoin (GJ) that captured both of the previous worst-case optimal algorithms.

GHDs. First introduced by Gottlob, Leone, and Scarcello [10], hypertree decompositions and the associated hypertree width generalize the concept of tree decompositions [25]. Conceptually, the decompositions capture a hypergraph’s cyclicity, allowing them to facilitate the selective use of GJ and Yannakakis in the standard hybrid join algorithm GHDJoin. There are deep connections between variable orderings and GHDs [16], which we leverage extensively. Grohe and Marx [12] introduced the idea of fractional hypertree width over GHDs, which bounds the runtime of GHDJoin by $\tilde{O}(\text{IN}^w + \text{OUT})$ ($\tilde{O}$ hides poly-logarithmic factors) for w defined to be the minimum fractional hypertree width among all GHDs.

Semirings and Aggregations. Green, Karvounarakis, and Tannen developed the idea of annotations over a semiring [11]. Our notation for the annotations is superficially different from theirs, solely for notational convenience. We delve into more detail in Section 3. This also has been used as a mechanism to capture aggregation in probabilistic databases [24].

MPF. Aji and McEliece [4] defined the “Marginalize a Product Function” (MPF) problem, which is equivalent to the space of AJAR queries with only one aggregation operator. They showed that MPF generalizes a wide variety of important algorithms and problems, which also implies that AJAR queries are quite general. They also provided a message passing algorithm to solve MPF, which has since been refined [15]. We further improve runtimes over the state of the art.

Aggregate-Join Queries. There is a standard modification to Yannakakis to handle aggregations [29], but the classic analysis provides only a $O(\text{IN} \cdot \text{OUT})$ bound. Bakibayev, Kocisky, Olteanu, and Zavodny study aggregation-join queries in factorized databases [6], and later Olteanu and Zavodny connected factorized databases and GHDs/GHDJoin [22]. They develop the intuition that if output attributes are above non-output attributes, the $+\text{OUT}$ runtime is preserved; we use the same intuition to develop and analyze AggroGHDJoin, a variant to GHDJoin for aggregate-join queries.

Abo Khamis, Ngo, and Rudra present the “Functional Aggregate Query” (FAQ) problem [16], which is equivalent to AJAR. The FAQ/AJAR problems arose out of discussions between Ngo, Rudra, and Ré at PODS12 about how to extend the worst-case result to queries using aggregation and message passing via Green et al.’s semiring formulation. We originally worked jointly on the problem, but we developed substantially different approaches, and decided to split our work. We argue that the AJAR approach is simpler and yields better runtime guarantees. We also describe new complete characterizations as described above. These completeness results let us connect more easily to existing literature. The algorithm described here has been implemented in the related database engine EmptyHeaded [1]². This engine has run motif finding, pagerank, and single-source shortest path queries, faster than previous high-level approaches that take datalog-like queries as input.

A primary application of multiple aggregation operators is quantified conjunctive queries (QCQ) and the counting

variant, which can be expressed as AJAR queries over the semiring $(\mathbb{V}, \wedge)$ with aggregations involving both operators. Here, we follow FAQ’s idea to formulate this as a query with product aggregation. Chen and Dalmau [7] completely characterized the space of tractable QCQ by defining a notion of width that relies on variable orderings. Chen and Dalmau’s width definition includes a complete characterization of the permissible variable orderings for a QCQ instance. Their characterization is similar in spirit to the partial ordering we define in Section 4 that characterizes the space of valid GHDs for an AJAR query. However, their results are focused on tractability rather than the optimal runtime exponents; our characterization extends theirs and has improved runtime bounds.

3. DEFINING AJAR

We start by describing some background material needed to define the AJAR problem. After that, we formally define the AJAR problem and discuss our solution to it.

3.1 Background

We use the classic hypergraph representation for database schema and queries [2]. A hypergraph $\mathcal{H}$ is a pair $(\mathcal{V}_{\mathcal{H}}, \mathcal{E}_{\mathcal{H}})$, where $\mathcal{V}_{\mathcal{H}}$ is a non-empty set of *vertices* and $\mathcal{E}_{\mathcal{H}} \subseteq 2^{\mathcal{V}_{\mathcal{H}}}$ is a set of *hyperedges*. Each $A \in \mathcal{V}_{\mathcal{H}}$ is called an *attribute*. Each attribute has a corresponding *domain* $\mathcal{D}^A$.

- **Data** For each hyperedge $F \in \mathcal{E}_{\mathcal{H}}$, there is a corresponding relation $R_F \subseteq \prod_{A \in F} \mathcal{D}^A$; we use the notation $\mathcal{D}^F$ to denote the domain of the tuples $\prod_{A \in F} \mathcal{D}^A$.
- **Join Query** Given a set $\mathcal{E}_{\mathcal{H}}$ and a relation R_F for each $F \in \mathcal{E}_{\mathcal{H}}$, let $\mathcal{V}_{\mathcal{H}} = \cup_{F \in \mathcal{E}_{\mathcal{H}}} F$. The join query is written $\bowtie_{F \in \mathcal{E}_{\mathcal{H}}} R_F$ and is defined as

$$\left\{ t \in \mathcal{D}^{\mathcal{V}_{\mathcal{H}}} \mid \forall F \in \mathcal{E}_{\mathcal{H}} : \pi_F(t) \in R_F \right\}$$

We use n to denote the number of attributes $|\mathcal{V}_{\mathcal{H}}|$ and m to denote the number of relations $|\mathcal{E}_{\mathcal{H}}|$. IN denotes the sum of sizes of input relations in a query, and OUT denotes the output size.

The complexity analysis in the paper is largely concerned with *data complexity*, i.e., all of our runtimes will be expressed in terms of IN and OUT . In Section 5, we develop ways to reduce query complexity as well.

A *path* from $A \in \mathcal{V}_{\mathcal{H}}$ to $B \in \mathcal{V}_{\mathcal{H}}$ in a hypergraph $\mathcal{H}$ is a sequence of attributes, starting with A and ending with B , such that each consecutive pair of attributes in the sequence occur together in a hyperedge. The number of attributes in the sequence is the *length* of the path.

We now define a GHD of a hypergraph.

DEFINITION 2. Given a hypergraph $\mathcal{H} = (\mathcal{V}_{\mathcal{H}}, \mathcal{E}_{\mathcal{H}})$, a generalized hypertree decomposition is a pair $(\mathcal{T}, \chi)$ of a tree $\mathcal{T} = (\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}})$ and function $\chi : \mathcal{V}_{\mathcal{T}} \rightarrow 2^{\mathcal{V}_{\mathcal{H}}}$ such that

- For each relation $F \in \mathcal{E}_{\mathcal{H}}$, there exists a tree node $t \in \mathcal{V}_{\mathcal{T}}$ that covers the edge, i.e., $F \subseteq \chi(t)$.
- For each attribute $A \in \mathcal{V}_{\mathcal{H}}$, the tree nodes containing A , i.e., $\{t \in \mathcal{V}_{\mathcal{T}} \mid A \in \chi(t)\}$, form a connected subtree.

²We have been told that LogicBlox has implemented a similar algorithm recently, but their approach is not public. We shared our implementation with them several months ago.

The latter condition is called the “running intersection property”. The $\chi(t)$ sets are referred to as ‘bags’ of the GHD. GHDs are assumed to be ‘rooted’ trees, which imposes a top-down partial order on their nodes³. Leveraging this order, for any GHD $(\mathcal{T}, \chi)$ and attribute $A \in \mathcal{V}_{\mathcal{H}}$, we define $TOP_{\mathcal{T}}(A)$ to be the top-most node $v \in \mathcal{V}_{\mathcal{T}}$ such that $A \in \chi(v)$.

When each bag of a GHD consists of the attributes of a single relation, the GHD is also called a *join tree*. Joins over a join tree can be processed using Yannakakis’ algorithm [29]. The runtime of Yannakakis’ algorithm is $O(\text{IN} + \text{OUT})$.

GHDs can be interpreted as query plans for joins. Given a GHD, we first join the attributes in each bag using worst case optimal algorithms [20, 28] to get one intermediate relation per bag. The intermediate relations can then be joined using Yannakakis’ algorithm. We call this combined algorithm GHDJoin.

The runtime of GHDJoin can be expressed in terms of the *fractional hypertree width* of the GHD:

DEFINITION 3. Given a hypergraph $\mathcal{H} = (\mathcal{V}_{\mathcal{H}}, \mathcal{E}_{\mathcal{H}})$, a GHD $(\mathcal{T}, \chi)$, and an instance I over $\mathcal{H}$, the fractional hypertree width, denoted $fhw(\mathcal{T}, \mathcal{H}, I)$, is defined to be $\max_{t \in \mathcal{T}} \rho_t^*$ in which ρ_t^* is the optimal value of the following linear program defined for each $t \in \mathcal{V}_{\mathcal{T}}$:

$$\begin{aligned} & \text{Minimize } \sum_{F \in \mathcal{E}_{\mathcal{H}}} x_F \log_{\text{IN}}(|R_F|) \text{ such that} \\ & \forall A \in \chi(t) : \sum_{F: A \in F} x_F \geq 1, \forall F \in \mathcal{E}_{\mathcal{H}} : x_F \geq 0 \end{aligned}$$

The fractional hypertree width is just the AGM bound [5] placed on the bags. Thus $\text{IN}^{fhw(\mathcal{T}, \mathcal{H}, I)}$ is an upper bound on the sizes of the intermediate relations of GHDJoin. GHDJoin runs in time $\tilde{O}(\text{IN}^{fhw(\mathcal{T}, \mathcal{H}, I)} + \text{OUT})$ for join queries.

We note that GHDs are traditionally defined as a triple $(\mathcal{T}, \chi, \lambda)$ where the function $\lambda : \mathcal{V}_{\mathcal{T}} \rightarrow 2^{\mathcal{E}_{\mathcal{H}}}$ assigns relations to each bag. Then the linear program in the definition of fhw is applied to the relations in $\lambda(t)$ for each node t . We omit this function and implicitly assign *every* relation to each bag (so $\lambda(t) = \mathcal{E}_{\mathcal{H}}$ for all $t \in \mathcal{V}_{\mathcal{T}}$). Though this makes a difference for certain notions of width, it leaves the fractional hypertree width unchanged, as adding more relations to the linear program cannot make the objective value worse. We also note that omitting function λ makes GHDs equivalent to Tree Decompositions. We use the terminology of GHDs as all related works on joins use GHDs; since there is a simple mapping between the two, it makes no difference formally.

We also note that our definition of fractional hypertree width is a little different from that originally presented by Grohe and Marx [12]; the original definition of fhw does not use the relation sizes in the objective value of its linear program. Inspired by AGM [5], we add the relation sizes as weights to actually form a tighter bound on runtime. And the development of worst-case join algorithms allows us to achieve this faster runtime.

Annotated Relations. To define a general notion of aggregations, we look to relations annotated with semirings [11].

³For ordinary join queries (without aggregation) the choice of root can be arbitrary; it does not affect the complexity or the correctness of GHDJoin. However, this top-down partial order on the nodes will play a larger role when answering AJAR queries.

DEFINITION 4. A commutative semiring is a triple $(S, \oplus, \otimes)$ of a set S and operators $\oplus : S \times S \rightarrow S$, $\otimes : S \times S \rightarrow S$ where there exist $0, 1 \in S$ such that for all $a, b, c \in S$ the following properties hold:

- *Identity and Annihilation:* $a \oplus 0 = a$, $a \otimes 1 = a$, $0 \otimes a = 0$
- *Associativity:* $(a \oplus b) \oplus c = a \oplus (b \oplus c)$, $(a \otimes b) \otimes c = a \otimes (b \otimes c)$
- *Commutativity:* $a \oplus b = b \oplus a$, $a \otimes b = b \otimes a$
- *Distributivity:* $a \otimes (b \oplus c) = (a \otimes b) \oplus (a \otimes c)$

Suppose we have some domain $\mathbb{K}$ and an operator set $O = \{\oplus^1, \oplus^2, \dots, \oplus^k, \otimes\}$ such that 0 is the identity for each $\oplus^i \in O$ and $(\mathbb{K}, \oplus^i, \otimes)$ forms a commutative semiring for each i . We then define a relation with an annotation from $\mathbb{K}$ for each tuple.

DEFINITION 5. An annotated relation with annotations from $\mathbb{K}$, or a $\mathbb{K}$ -relation, over attribute set F is a set $\{(t_1, \lambda_1), (t_2, \lambda_2), \dots, (t_N, \lambda_N)\}$ such that for all $1 \leq i \leq N$, $t_i \in \mathcal{D}^F$, $\lambda_i \in \mathbb{K}$ and for all $1 \leq j \leq N : i \neq j \rightarrow t_i \neq t_j$.

Green et al. define a $\mathbb{K}$ -relation to be a function $R_F : \mathcal{D}^F \rightarrow \mathbb{K}$ [11]. Our notion can be viewed as an explicit listing of this function’s support. But unlike an explicit listing of the function’s support, our table does allow tuples with 0 annotations. However, under our definitions of the operators below, an annotation of 0 is semantically equivalent to a tuple being absent (we discuss this further in Section 6). Note that we can have an annotated relation of the form $R_{\emptyset}$ of size 1 containing the empty tuple with some annotation. We now define joins and aggregations over annotated relations.

Joins over Annotated Relations. Informally, a join over annotated relations is obtained as follows. We first perform a regular join on the non-annotated part of the relations. Then, for each output tuple t of the join, we set its annotation to the product of the annotations of the input tuples used to produce t . Formally, we define a join $\bowtie_{F \in \mathcal{E}_{\mathcal{H}}} R_F$ as:

$$\{(t, \lambda) \mid t \in \mathcal{D}^{\mathcal{V}_{\mathcal{H}}}, \lambda = \prod_{F \in \mathcal{E}_{\mathcal{H}}} \lambda_F : (\pi_F(t), \lambda_F) \in R_F\}$$

Aggregations over Annotated Relations. An aggregation over an annotated relation R_F is specified by a pair $(A, \oplus)$ where $A \in F$, and $\oplus \in O$. The aggregation takes groups of tuples in R_F that share values of all attributes other than A , and produces a single tuple corresponding to each group, whose annotation is the $\oplus$ -aggregate of the annotations of the tuples in the group. Suppose that R has schema $R(A, B)$ in which A is a single attribute and B is a set of attributes. Then, the result of aggregation $(A, \oplus)$ has only the attributes B and

$$\sum_{(A, \oplus)} R_{A, B} = \{(t_B, \lambda) \mid t_B \in \pi_B R \text{ and } \lambda = \bigoplus_{(t, \lambda_t) \in R : \pi_B t = t_B} \lambda_t\}$$

One can define the meaning of aggregate-join queries in a straightforward way: first compute the join and then perform aggregations. Figure 2 shows some examples of operators on relations. For the remainder of our work, we assume that all relations are $\mathbb{K}$ -relations.

R			S			R ⋈ S			
A	B	⊕	B	C	⊕	A	B	C	⊕
1	1	1	1	1	3	1	1	1	3
2	1	2	1	2	4	1	1	2	4
						2	1	1	6
						2	1	2	8

$\Sigma_{(C,+)} S$		
A	B	⊕
1	1	7
2	1	14

Figure 2: Selected examples illustrating the operators over the semiring $(\mathbb{R}_+, +, \cdot)$

3.2 The AJAR problem

DEFINITION 6. Given some global attribute set $\mathcal{V}$ and operator set O , we define an aggregation ordering to be a sequence $\alpha = \alpha_1, \alpha_2, \dots, \alpha_s$ such that for each $1 \leq i \leq s$, $\alpha_i = (a_i, \oplus_i)$ for some $a_i \in \mathcal{V}, \oplus_i \in O$.⁴ In addition, attributes occur at most once, i.e., $a_j \neq a_k$ for each $1 \leq j < k \leq s$.

Informally, the aggregation ordering is a sequence of attribute-operator pairs such that each attribute appears at most once. Note that the aggregation ordering specifies the order and manner in which attributes are aggregated. The ordering does not need to contain every attribute; we use the term *output attributes* to denote the attributes not in the ordering.

$V(\alpha)$ represents the set of attributes that appear in α , and $V(-\alpha)$ represents $\mathcal{V} \setminus V(\alpha)$, i.e., the output attributes. When $F \subseteq V(\alpha)$, we use α_F to denote the restriction of α to the attributes in F , so $V(\alpha_F) = F$.

DEFINITION 7 (AJAR). Given some hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and an aggregation ordering α , an AJAR query $Q_{\mathcal{H}, \alpha}$ is a function over instances of $\mathcal{H}$ such that

$$Q_{\mathcal{H}, \alpha}(\{R_F | F \in \mathcal{E}\}) = \Sigma_{\alpha_1} \cdots \Sigma_{\alpha_{|\alpha|}} \bowtie_{F \in \mathcal{E}} R_F.$$

3.3 Answering AJAR Queries

For an AJAR query, we define OUT to be the final output size, which is smaller than the output size of the join when $|\alpha| > 0$. Thus the standard GHDJoin runtime of $\text{IN}^{fhw} + \text{OUT}$ is harder to achieve. In this section, we achieve this standard runtime even with aggregations by restricting the class of acceptable GHDs.

We present a simple variant of GHDJoin, called AggroGHDJoin, that performs aggregations on annotated relations.

Algorithm 1 AggroGHDJoin($\mathcal{H} = (\mathcal{V}_{\mathcal{H}}, \mathcal{E}_{\mathcal{H}})$, α , $(\mathcal{T}(\mathcal{V}_{\mathcal{T}}, \mathcal{E}_{\mathcal{T}}), \chi)$, $\{R_F | F \in \mathcal{E}_{\mathcal{H}}\}$)

Input: Query hypergraph $\mathcal{H}$, Aggregation Ordering α , GHD $(\mathcal{T}, \chi)$, Relations R_F for each $F \in \mathcal{E}_{\mathcal{H}}$

```

 $S_R \leftarrow \emptyset$ 
for all  $t \in \mathcal{V}_{\mathcal{T}}$  do
   $\mathcal{H}_t \leftarrow (\chi(t), \{\pi_{\chi(t)} F | F \in \mathcal{E}_{\mathcal{H}}\})$ 
   $I \leftarrow \{R_F | F \subseteq \chi(t), \exists a \in F : \text{TOP}_{\mathcal{T}}(a) = t\} \cup$ 
   $\{\pi_{\chi(t)}^1 R_F | F \not\subseteq \chi(t) \text{ or } \forall a \in F : \text{TOP}_{\mathcal{T}}(a) \neq t\}$ 
   $S_R \leftarrow S_R \cup \text{GJ}(\mathcal{H}_t, I)$ 
end for
return AggroYannakakis( $\mathcal{T}, \alpha, S_R$ )

```

⁴Note that, by this definition, the operators in aggregation ordering can be the product aggregation $\otimes$. However, product aggregations require different definitions, see Section 6.

The operator π^1 denotes projection that sets the annotations of all output tuples to 1. We require this operator to construct the instances I passed to GJ to ensure the annotations of a particular relation only get multiplied once in the join. AggroYannakakis is a slightly modified version of the classic project-join algorithm by Yannakakis [29], with the modifications necessary to handle the annotations correctly; we provide the details in Appendix A.

We start by reasoning about the correctness of AggroGHDJoin. We note that AJAR queries are defined with aggregation orderings, but our algorithm uses GHDs instead of orderings. We now connect the two notions.

DEFINITION 8. Given an AJAR query $Q_{\mathcal{H}, \alpha}$, a GHD $(\mathcal{T}, \chi)$ of $\mathcal{H}$ is compatible with α if and only if $\text{TOP}_{\mathcal{T}}(A)$ being an ancestor of $\text{TOP}_{\mathcal{T}}(B)$ implies A either precedes B in α or is an output attribute.

THEOREM 9. Algorithm 1 correctly computes an AJAR query $Q_{\mathcal{H}, \alpha}$ if it is given a GHD $(\mathcal{T}, \chi)$ of $\mathcal{H}$ that is compatible with α .

PROOF. (Sketch) Note that Yannakakis (and AggroYannakakis) performs the projections (i.e., aggregations) in a bottom-up manner. Moreover, a GHD is compatible with a given ordering iff some top-down ordering of the attributes is equivalent to the given ordering. $\square$

Suppose our given GHD has fhw w . Traditional analysis of GHDJoin and GJ tells us the portion of AggroGHDJoin before calling AggroYannakakis runs in $\tilde{O}(\text{IN}^w)$. However, the traditional analysis of Yannakakis on project-join queries only provides a guarantee of $O(\text{IN} \cdot \text{OUT})$. The underlying cause for this blowup is that the $\text{TOP}_{\mathcal{T}}$ nodes for output attributes may occur below the $\text{TOP}_{\mathcal{T}}$ nodes of non-output attributes. We note that our definition ensures this never happens in any compatible GHD, avoiding this blowup.

THEOREM 10. Given an AJAR query $Q_{\mathcal{H}, \alpha}$, let w^* denote the smallest fhw for a GHD compatible with α ; the runtime of our approach is $\tilde{O}(\text{IN}^{w^*} + \text{OUT})$.

We trace through the details of the proof in Appendix A, but it follows the spirit of the proofs of the classic Yannakakis algorithm.

4. CHARACTERIZING EQUIVALENT ORDERINGS

We discussed an algorithm to solve AJAR queries with a seemingly optimal (or near-optimal) runtime of $\tilde{O}(\text{IN}^{w^*} + \text{OUT})$. However, note that w^* is optimal width over an extremely restrictive class of GHDs. In this section we expand the class of permissible GHDs by identifying alternative aggregation orders that produce the same output as α over any database instance; some of these new orders may allow for faster runtimes, e.g., some orders may allow early aggregation.

As suggested above, we can define a natural equivalence relation over aggregation orderings.

DEFINITION 11 (EQUIVALENT ORDERINGS). Given a hypergraph $\mathcal{H}$, define the equivalence relation between orderings $\equiv_{\mathcal{H}}$ such that $\alpha \equiv_{\mathcal{H}} \beta$ if and only if $Q_{\mathcal{H}, \alpha}(I) = Q_{\mathcal{H}, \beta}(I)$ for all database instances I over the schema $\mathcal{H}$.

To determine which orders are equivalent, we first investigate which attribute-operator pairs commute. We make two simple observations: (1) by the Eckmann-Hilton argument [8], two operators commute in general if and only if they are equal⁵ and (2) aggregations can be pushed down into joins, which means two attributes commute if they are in independent parts of the join. We illustrate these forms of commuting by example and then formalize them.

EXAMPLE 12. Suppose we have the AJAR query $Q_{\mathcal{H},\alpha} = \sum_{(A,+)} \sum_{(B,+)} \sum_{(C,\max)} R(A,B) \bowtie S(A,C)$. The first pair of aggregations can commute since they have identical operators; if $\beta = ((B,+), (A,+), (C,\max))$, $\alpha \equiv_{\mathcal{H}} \beta$. Similarly the second pair of aggregations can commute because the projection can be pushed down:

$$\begin{aligned} & \sum_{(B,+)} \sum_{(C,\max)} R(A,B) \bowtie S(A,C) \\ &= \left[\sum_{(B,+)} R(A,B) \right] \bowtie \left[\sum_{(C,\max)} S(A,C) \right] \\ &= \sum_{(C,\max)} \sum_{(B,+)} R(A,B) \bowtie S(A,C) \end{aligned}$$

So if $\beta = ((A,+), (C,\max), (B,+))$, $\alpha \equiv_{\mathcal{H}} \beta$.

THEOREM 13. Suppose we are given a relation R_F such that $A, B \in F$ and two operators $\oplus', \oplus \in O$. Then

$$\Sigma_{(A,\oplus)} \Sigma_{(B,\oplus')} R_F = \Sigma_{(B,\oplus')} \Sigma_{(A,\oplus)} R_F$$

if one of the following conditions hold:

- $\oplus = \oplus'$
- There exist relations R_{F_1} and R_{F_2} such that $A \notin F_1$, $B \notin F_2$, and $R_{F_1} \bowtie R_{F_2} = R_F$.

The proof of this theorem is straightforward, but it provides the foundation for an algorithm to test the equivalence of two aggregation orderings. This testing algorithm combined with a modified join algorithm (discussed next) lets us recover and surpass runtimes of previous work (see our technical report [13] for details).

Note that the equivalence relation $\equiv_{\mathcal{H}}$ defines equivalence classes among the orderings that may be exponential in size. We are motivated to find a more succinct characterization that lets us enumerate all equivalent orderings with constant delay. Chen and Dalmau [7] obtained a similar order-equivalence characterization for a special case of the AJAR problem, namely for aggregations “and” and “or”. The characterization was based on a procedure that generated all equivalent orderings. We provide a succinct characterization of the equivalence class of an aggregation ordering with any number of operators⁶.

To that end, we develop an enumeration of the constraints that are sufficient and necessary for an ordering to be in the equivalence class of α . The constraints are of the form “ A must always occur before B ”:

⁵Two operators $\oplus, \oplus'$ are equal over a domain $\mathbb{K}$ if $\forall x, y \in \mathbb{K}$, $x \oplus y = x \oplus' y$

⁶“and” is a product aggregation, so the generalization of Chen and Dalmau’s characterization is only covered in Section 6

DEFINITION 14 (PREC). Given an AJAR query $Q_{\mathcal{H},\alpha}$, define a constraint $PREC \subseteq \mathcal{V} \times \mathcal{V}$ such that $(A, B) \in PREC$ if and only if A precedes B in all orderings that are equivalent to α .

We say $PREC(A, B)$ is true if and only if $(A, B) \in PREC$.

Trivially, the number of pairs in $PREC$ is less than n^2 . We note that we can use $PREC$ to define a (strict) partial ordering on the attributes; the constraints are clearly anti-reflexive, antisymmetric, and transitive. We use $<_{\mathcal{H},\alpha}$ to denote this partial order. Given an AJAR query $Q_{\mathcal{H},\alpha}$, $<_{\mathcal{H},\alpha}$ is a partial order of attribute-operator pairs such that for any $(A, \oplus), (B, \oplus') \in \alpha$, $(A, \oplus) <_{\mathcal{H},\alpha} (B, \oplus')$ if $PREC(A, B)$ (see Definition 23 for the exact definition). The partial order $<_{\mathcal{H},\alpha}$ is easier to use for proofs; we use the partial order to show the soundness and completeness of these constraints.

THEOREM 15 ($<_{\mathcal{H},\alpha}$ SOUNDNESS AND COMPLETENESS). Suppose we are given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and aggregation orderings α, β . Then $\alpha \equiv_{\mathcal{H}} \beta$ if and only if β is a linear extension of $<_{\mathcal{H},\alpha}$.

We first describe a procedure to compute the precedence relation $PREC$. After that, we reason about its completeness.

Computing $PREC$. To assist in building $PREC$, we define a constraint of the form “ A and B cannot commute”:

DEFINITION 16 (DNC). Given an AJAR query $Q_{\mathcal{H},\alpha}$, define a constraint $DNC \subseteq \mathcal{V} \times \mathcal{V}$ such that $(A, B) \in DNC$ if and only if A and B are in the same order in any β such that $\beta \equiv_{\mathcal{H}} \alpha$.

Once again, we say $DNC(A, B)$ is true if and only if $(A, B) \in DNC$. We prefer to work with DNC because we have already discussed when aggregations can commute in Theorem 13; the conditions of that theorem specify when DNC is *FALSE*. However, we can immediately derive a simple relationship between $PREC$ and DNC :

LEMMA 17. Given an AJAR query $Q_{\mathcal{H},\alpha}$, for any $A, B \in \mathcal{V}$, $PREC(A, B)$ iff $DNC(A, B)$ and A precedes B in α .

We now develop conditions when DNC is true. Recall that Theorem 13 states that two aggregations can commute if (1) they have the same operator or (2) they can be separated in the join query; the simplest structure that violates both of these conditions is an edge that contains two attributes with differing aggregating operators.

LEMMA 18. Given an AJAR query $Q_{\mathcal{H},\alpha}$, suppose $(A, \oplus), (B, \oplus') \in \alpha$. If $\oplus \neq \oplus'$ and there exists an edge $E \in \mathcal{E}$ such that $A, B \in E$, then $DNC(A, B)$.

Lemma 18 serves as a base case, but we want to extend the violation of Theorem 13’s conditions beyond single edges to paths. To do so, consider the following examples of how our commuting conditions interact with paths of length two.

EXAMPLE 19. Consider the query

$$\sum_{(A,+)} \sum_{(B,\max)} \sum_{(C,\max)} R(A,B) \bowtie S(B,C).$$

No two attributes can be separated, which implies $DNC(A, B)$ and $DNC(A, C)$. Lemma 18 gives us the former constraint,

but not the latter one. This example indicates that it may be possible to extend a constraint $DNC(A, B)$ along an edge $\{B, C\}$. On the other hand, consider the query

$$\sum_{(B, \max)} \sum_{(A, +)} \sum_{(C, \max)} R(A, B) \bowtie S(B, C).$$

Note that A and C can be separated, which implies that only $DNC(A, B)$ holds. Note that, as before, Lemma 18 gives us this constraint. This example suggests that we cannot extend every $DNC(A, B)$ constraint along an additional edge.

The key difference between the two examples is the *relative order* of A and B in α , which suggests that we can only extend $DNC(A, B)$ along an edge if A precedes B in α , i.e., if $PREC(A, B)$.

LEMMA 20. *Given an AJAR query $Q_{\mathcal{H}, \alpha}$, suppose $(A, \oplus), (B, \oplus') \in \alpha$. If $\oplus \neq \oplus'$ and $\exists C \in \mathcal{V}, E \in \mathcal{E} : PREC(A, C)$ and $B, C \in E$, then $DNC(A, B)$.*

$PREC$ is transitive, which implies:

LEMMA 21. *Given an AJAR query $Q_{\mathcal{H}, \alpha}$, suppose $(A, \oplus), (B, \oplus') \in \alpha$. If $\exists C : PREC(A, C)$ and $PREC(C, B)$, then $DNC(A, B)$.*

The above transitivity condition interacts with the condition from Lemma 20 in interesting ways.

EXAMPLE 22. *Consider the query,*

$$\sum_{(A, +)} \sum_{(B, \max)} \sum_{(C, \max)} \sum_{(D, +)} R(A, B) \bowtie S(B, D) \bowtie T(C, D).$$

No attributes can be separated, which implies $DNC(A, B)$, $DNC(A, C)$, $DNC(B, D)$, and $DNC(C, D)$. Transitivity gives $DNC(A, D)$ as well. Now let us derive these constraints using Lemmas 18, 20, and 21. Lemma 18 gives us $DNC(A, C)$, $DNC(B, D)$, and $DNC(C, D)$. Note that at this point, Lemma 20 gives us no more constraints. Only after the transitivity of Lemma 21 adds the constraint $DNC(A, D)$ can Lemma 20 add the constraint $DNC(A, B)$, completing the set of constraints.

It turns out that these three simple lemmas provide sufficient and necessary constraints on the equivalence classes of orderings; no other conditions are necessary to complete the proofs the soundness and completeness of $<_{\mathcal{H}, \alpha}$.

We note that our current specifications of $PREC$ and DNC are mutually recursive. The $PREC$ and DNC sets build up in rounds; Lemma 18 provides their initial values, and Lemmas 17, 20, and 21 iteratively build up the sets further. We keep applying these lemmas until the sets reach a fixed point. This takes at most $2|\alpha|^2$ iterations, as we must add at least one additional attribute pair per iteration, and there can be only $|\alpha|^2$ pairs of attributes in each set. Thus the overall runtime of computing these constraints is polynomial in the number of attributes. Please see our technical report for more details [13].

For convenience of notation, we make one modification to the definition of the partial order $<_{\mathcal{H}, \alpha}$. When A is an output attribute and B is not, we define $A <_{\mathcal{H}, \alpha} B$ to be true. So we can formally state the definition as:

DEFINITION 23 ($<_{\mathcal{H}, \alpha}$). *Given a AJAR query $Q_{\mathcal{H}, \alpha}$, we define $A <_{\mathcal{H}, \alpha} B$ to be true if either (i) A is an output attribute and B is not or (ii) $PREC(A, B)$ is true.*

Soundness and Completeness of $<_{\mathcal{H}, \alpha}$. To prove the soundness and completeness of $<_{\mathcal{H}, \alpha}$, we now state two key lemmas illustrating properties of $<_{\mathcal{H}, \alpha}$. We mostly provide proof sketches, please see our technical report [13] for full proofs.

LEMMA 24. *Suppose we are given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and an aggregation ordering α . Suppose $(A, \oplus), (B, \oplus') \in \alpha$ for differing operators $\oplus \neq \oplus'$. Then, for any path P in $\mathcal{H}$ between A and B , there must exist some attribute in the path $C \in P$ such that $C <_{\mathcal{H}, \alpha} A$ or $C <_{\mathcal{H}, \alpha} B$.*

Note that the C in the statement above can be either A or B . Lemma 24 is only non-trivial if A and B are incomparable. Intuitively the lemma states that attributes incomparable under $<_{\mathcal{H}, \alpha}$ with differing operators must be separated in $\mathcal{H}$ by their combined predecessors in $<_{\mathcal{H}, \alpha}$.

PROOF. (Sketch) We proceed by induction on the length of the path P . We let C be the attribute between A and B that appears earliest in α and assume without loss of generality that A and C have differing operators. By the inductive hypothesis, there must exist a D on the path from A to C such that either $D <_{\mathcal{H}, \alpha} A$ or $D <_{\mathcal{H}, \alpha} C$. In the former case, our proof is done. In the latter case we reason that since C is defined to be the attribute that appears earliest in α between A and B , $D <_{\mathcal{H}, \alpha} C$ implies $D = A$. We then prove by induction that $A <_{\mathcal{H}, \alpha} X$ for all X on the path between C and B , which then implies that A and B cannot be incomparable. $\square$

This lemma allows us to prove the soundness of $<_{\mathcal{H}, \alpha}$.

LEMMA 25. (Soundness of $<_{\mathcal{H}, \alpha}$) *If β is a linear extension of $<_{\mathcal{H}, \alpha}$, then $\alpha \equiv \beta$.*

PROOF. We induct on the number of inversions in β with respect to the ordering α . Our base case, 0 inversion, is trivial. Now suppose β has $N > 0$ inversions, and assume the lemma is true for orderings with $< N$ inversions. There must be some β_i and β_{i+1} that are inverted with respect to α . Consider the ordering β' derived by swapping β_i and β_{i+1} . By the inductive hypothesis, $\alpha \equiv_{\mathcal{H}} \beta'$.

We now show that $\beta \equiv_{\mathcal{H}} \beta'$. Suppose $\beta_i = (A, \oplus)$ and $\beta_{i+1} = (B, \oplus')$. We have two cases to consider.

- $\oplus = \oplus'$
By Theorem 13, we can swap β_i and β_{i+1} without affecting the output. This implies that $\beta \equiv_{\mathcal{H}} \beta'$.
- $\oplus \neq \oplus'$
By Lemma 24, any path between A and B must go through some attribute C that appears earlier than index i in β . This implies that A and B are in separate connected components in $\pi_{V(\beta_i, \beta_{i+1}, \dots, \beta_{|\beta|})} \mathcal{H}$, which implies that we can swap β_i and β_{i+1} without affecting the output by Theorem 13. Thus $\beta \equiv_{\mathcal{H}} \beta'$.

$\square$

Our next lemma helps us prove the completeness of $<_{\mathcal{H}, \alpha}$.

LEMMA 26. *Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and an aggregation ordering α , suppose $A <_{\mathcal{H}, \alpha} B$ for some $A, B \in V(\alpha)$. Then there must exist a path P from A to B such that for every $C \in P, C \neq A$ we have $A <_{\mathcal{H}, \alpha} C$.*

PROOF. (Sketch) We proceed by induction on the recursive definition of $<_{\mathcal{H},\alpha}$. Lemma 18 implies A and B share an edge. Lemma 20 implies the existence of a C such that either $A <_{\mathcal{H},\alpha} C, (B, C) \in \mathcal{E}$ or $B <_{\mathcal{H},\alpha} C, (A, C) \in \mathcal{E}$. Either way we inductively construct a path between C and the attribute it doesn't share an edge with. Finally, Lemma 21 implies the existence of a C such that $A <_{\mathcal{H},\alpha} C <_{\mathcal{H},\alpha} B$, which lets us inductively construct paths from A to C and C to B . $\square$

LEMMA 27. (Completeness of $<_{\mathcal{H},\alpha}$) If β is not a linear extension of $<_{\mathcal{H},\alpha}$, then $\alpha \not\equiv \beta$.

PROOF. (Sketch) We first construct a particular ordering $\alpha' \equiv \alpha$ that allows us to find two attribute-ordering pairs $(A, \oplus), (B, \oplus') \in \alpha'$ such that $B <_{\mathcal{H},\alpha} A, \oplus \neq \oplus'$, and A precedes B in β . Since $\oplus \neq \oplus'$, there must exist $x, y \in \mathbb{K}$ such that $x \oplus y \neq x \oplus' y$. Our proof constructs an instance $\hat{I}$ such that $Q_{\mathcal{H},\alpha}(\hat{I})$ returns one tuple with annotation $x \oplus' y$ and $Q_{\mathcal{H},\beta}(\hat{I})$ returns one tuple with annotation $x \oplus y$.

Our construction relies on Lemma 26, which implies the existence of a path P from A to B such that every attribute in our path comes after B in α' . Additionally, by construction, we can guarantee that every attribute in our path comes after A in β . We then finish by constructing an instance so that only the outermost aggregation of the path P changes the relation; since the outermost aggregations of α and β have different operators, we get different annotations in the output. $\square$

5. DECOMPOSING VALID GHDs

We express our AJAR algorithm directly in terms of GHDs, rather than in terms of aggregation orderings. As such, our goal is the characterization of GHDs that are compatible with at least one equivalent ordering, i.e., the GHDs that can be used to answer an AJAR query. We call a GHD *valid* if it is compatible with at least one equivalent ordering. We first give a simple characterization of valid GHDs. Then we demonstrate a way to reduce the problem of finding a minimum-width valid GHD to multiple subproblems on unconstrained GHDs (Section 5.1). This decomposition of the problem gets us three things:

- We can speed up our brute force search for an optimal valid GHD. We can also find approximately optimal valid GHDs in polynomial time using Marx's GHD approximation algorithm [17] (Section 5.2).
- We can apply existing MapReduce join algorithms that utilize GHDs [3], obtaining efficient parallel algorithms for solving AJAR queries (Section 5.4).
- We can apply improved join algorithms [14, 18] to further reduce our runtime exponent (Section 5.3).

5.1 Valid and Decomposable GHDs

We can easily characterize valid GHDs by combining the definition of compatible GHDs with Theorem 15.

THEOREM 28. For an AJAR query $Q_{\mathcal{H},\alpha}$, a GHD $(\mathcal{T}, \chi)$ is valid if and only if for every pair of attributes A, B such that $TOP_{\mathcal{T}}(A)$ is an ancestor of $TOP_{\mathcal{T}}(B)$, $B \not\prec_{\mathcal{H},\alpha} A$.

Theorem 28 gives us a criterion specifying which GHDs can act as query plans. We now consider the problem of finding

a minimum width valid GHD for any AJAR query. We call a GHD *optimal* if it has the minimum width possible for valid GHDs. We show how to reduce the problem of finding an optimal valid GHD into smaller problems of finding ordinary optimal GHDs. This unlocks a trove of powerful GHD results and makes them applicable to our problem.

Given an AJAR query $Q_{\mathcal{H},\alpha}$, suppose we have a subset of the nodes $V \subseteq \mathcal{V}$. Define $\mathcal{E}_V$ to be $\{E \in \mathcal{E} \mid E \cap V \neq \emptyset\}$, i.e., the set of edges that intersect with V . As before, α_V denotes the aggregation ordering restricted to the nodes in V . Additionally define V^O to be $\{v \in V \mid \forall w \in V, w \not\prec_{\mathcal{H},\alpha} v\}$, i.e., the nodes in V that have no predecessors in V according to the partial ordering $<_{\mathcal{H},\alpha}$. Note that $\alpha_{V \setminus V^O}$ is then α_V with all the nodes in V^O removed (this makes the nodes in V^O output attributes). Finally, for a GHD $(\mathcal{T}, \chi)$, we will use $\chi(\mathcal{T})$ to denote $\cup_{t \in \mathcal{V}_{\mathcal{T}}} \chi(t)$.

DEFINITION 29. Given an AJAR query $Q_{\mathcal{H},\alpha}$, we say a GHD $(\mathcal{T}, \chi)$ is decomposable if:

- There exists a rooted subtree $\mathcal{T}_0$ of $\mathcal{T}$ such that $\chi(\mathcal{T}_0) = \mathcal{V}(-\alpha)$ (i.e., output attributes).
- For each connected component C of $\mathcal{H} \setminus V_{-\alpha}$, there is exactly one subtree $\mathcal{T}_C \in \mathcal{T} \setminus \mathcal{T}_0$ such that $\mathcal{T}_C$ is a decomposable GHD of $Q_{(\cup_{E \in \mathcal{E}_C} E, \mathcal{E}_C), \alpha_{C \setminus C^O}}$.

We start by connecting this idea of decomposable GHDs to valid GHDs. We only give proof sketches here; see our technical report for the full proofs.

THEOREM 30. Every decomposable GHD is valid.

PROOF. (Sketch) Suppose the AJAR query is $Q_{\mathcal{H},\alpha}$. We need to show for any A, B such that $TOP_{\mathcal{T}}(A)$ is an ancestor of $TOP_{\mathcal{T}}(B)$, $A \not\prec_{\mathcal{H},\alpha} B$. We use induction on $|\alpha|$. If $|\alpha| = 0$, all GHDs are valid and decomposable. For $|\alpha| > 0$, $\mathcal{T}_0$ ensures that the output attributes are above non-output attributes. If A and B are non-output attributes and $TOP_{\mathcal{T}}(A)$ is an ancestor of $TOP_{\mathcal{T}}(B)$, then both are in some $\mathcal{T}_C$. By the inductive hypothesis, $\mathcal{T}_C$ is valid with respect to $Q_{(\cup_{E \in \mathcal{E}_C} E, \mathcal{E}_C), \alpha_{C \setminus C^O}}$. By inspecting the partial order created by this subgraph, we conclude that $A \not\prec_{\mathcal{H},\alpha} B$ as desired. $\square$

Not every valid GHD is decomposable. However, every valid GHD can be transformed into a corresponding decomposable GHD using some simple transformations. Each bag of the resulting decomposable GHD is a subset of one of the bags of the original GHD. Thus the *fhw* of the decomposable GHD is at most the *fhw* of the original valid GHD. In fact, we can make a more general claim, using a notion of node-monotone functions, defined next.

DEFINITION 31. Given a hypergraph $\mathcal{H} = (\mathcal{V}_{\mathcal{H}}, \mathcal{E}_{\mathcal{H}})$, we define a function to be node-monotone if it is a function $\gamma : 2^{\mathcal{V}_{\mathcal{H}}} \rightarrow \mathbb{R}$ such that $\forall A \subseteq B \subseteq \mathcal{V}_{\mathcal{H}} : \gamma(A) \leq \gamma(B)$. Given any node-monotone function γ , we define the γ -width of a GHD $(\mathcal{T}, \chi)$ over $\mathcal{H}$ as $\max_{v \in \mathcal{V}_{\mathcal{T}}} \gamma(\chi(v))$.

Many standard notions of widths can be expressed as γ -widths for a suitably chosen γ . Specifically:

PROPOSITION 1. Suppose we are given a hypergraph $\mathcal{H} = (\mathcal{V}_{\mathcal{H}}, \mathcal{E}_{\mathcal{H}})$ and database instance I on $\mathcal{H}$. Then for each of following notions of width: (i) Treewidth (ii) Generalized Hypertree Width (iii) Fractional Hypertree Width (iv) Submodular Width, there exists a node-monotone function γ such that γ -width equals the given notion of width.

As a simple example, tree-width can be expressed as γ -width for $\gamma(A) = |A| - 1$. We can now relate valid and decomposable GHDs with respect to their γ -widths.

THEOREM 32. *For every valid GHD $(\mathcal{T}, \chi)$, there exists a decomposable GHD $(\mathcal{T}', \chi')$ such that for all node-monotone functions γ , the γ -width of $(\mathcal{T}', \chi')$ is no larger than the γ -width of $(\mathcal{T}, \chi)$.*

PROOF. (Sketch) Suppose the AJAR query is $Q_{\mathcal{H}, \alpha}$. We transform the given GHD $(\mathcal{T}, \chi)$ into $(\mathcal{T}', \chi')$ such that for each $v' \in \mathcal{V}_{\mathcal{T}'}$, there exists a $v \in \mathcal{V}_{\mathcal{T}}$ such that $\chi'(v') \subseteq \chi(v)$. The result then follows from the node-monotonicity of γ and the definition of γ -width. Any transformation of a GHD that ensures that all new bags are subsets of old bags, is called width-preserving.

We then transform the GHD $(\mathcal{T}, \chi)$ to satisfy the following properties (we describe the corresponding width-preserving transformations):

- Every $t \in \mathcal{T}$ is $TOP_{\mathcal{T}}(A)$ for exactly one attribute A .
We can remove every node $t \in \mathcal{T}$ that is the top node for no attributes. For a node t that is the top node for multiple attributes, we create a new node t' and set it to be the parent of t such that $\chi(t') \subseteq \chi(t)$. We specify $\chi(t')$ such that t is the top node for one fewer attribute than before. We can repeat this process until our condition is met.
- For any node $t \in \mathcal{T}$ and the subtree $\mathcal{T}_t$ rooted at t , the attributes $\{v \in \mathcal{V} | TOP_{\mathcal{T}}(v) \in \mathcal{T}_t\}$ form a connected subgraph of $\mathcal{H}$.

We transform the GHD to ensure this property in a bottom-up manner. In particular, we only consider a node $t \in \mathcal{T}$ after we know its children all satisfy the property. Then if any child's subtree is not connected to t in $\mathcal{H}$, we can safely connect the child to t 's parent. This process ensures that t satisfies our property.

We can show, by induction, any valid GHD that satisfies these two properties is decomposable. Intuitively, the first transformation ensures the subtree $\mathcal{T}_0$ exists as desired. The second transformation ensures that each of the $\mathcal{T}_C$'s exists and satisfies the requisite properties. $\square$

This theorem lets us restrict our search to the smaller space of decomposable GHDs (instead of all valid GHDs) when looking for the optimal valid GHD. Moreover, the space of decomposable GHDs is simpler; it can be factored into smaller spaces of unconstrained GHDs, as we show next. We present the definition of *characteristic hypergraphs*, which are intuitively the set of hypergraphs that specify the factors, i.e., the unconstrained GHDs.

Our goal is two-fold: (1) to be able to split a decomposable GHD into component GHDs of the characteristic hypergraphs and (2) to be able to take arbitrary GHDs of the characteristic hypergraphs and connect them to create a decomposable GHD of the original AJAR problem. The definition of decomposable GHDs decomposes a GHD into a series of sub-trees $\mathcal{T}_0, \dots, \mathcal{T}_k$. The definition specifies that the subtrees $\mathcal{T}_1, \dots, \mathcal{T}_k$ must be decomposable GHDs of (smaller) AJAR problems. Additionally, it is simple to show $\mathcal{T}_0$ is a GHD of the hypergraph $(V(-\alpha), \{E \in \mathcal{E} | E \subseteq V(-\alpha)\})$. If we apply this decomposition recursively to the subtrees

$\mathcal{T}_1, \dots, \mathcal{T}_k$, we can divide any decomposable GHD into a series of (unrestricted) GHDs of particular hypergraphs. This provides the basis of our definition of the characteristic hypergraphs; we define a hypergraph $\mathcal{H}_0$ that specifies the hypergraph corresponding to $\mathcal{T}_0$ and then recurse on the smaller AJAR queries specified in Definition 29.

However, if we are given arbitrary GHDs of the hypergraphs as defined thus far, we may not be able to stitch them together while preserving the running intersection property of GHDs. To ensure this stitching is possible, we need the characteristic hypergraphs to contain additional edges that we can use to guarantee the running intersection property. Intuitively the edges we add will be the intersections of the adjacent subtrees in our decomposition; for example, for any connected component C of $\mathcal{H} \setminus V(-\alpha)$, $\mathcal{T}_0$ and $\mathcal{T}_C$ are adjacent, and we will add the edge $\chi(\mathcal{T}_0) \cap \chi(\mathcal{T}_C)$ to the corresponding hypergraphs. We can use these 'intersection edges' to connect particular nodes in the adjacent subtrees.

DEFINITION 33. *Given an AJAR problem $Q_{\mathcal{H}, \alpha}$, suppose $C_1, \dots, C_k$ are the connected components of $\mathcal{H} \setminus V(-\alpha)$. Define a function $H(\mathcal{H}, \alpha)$ that maps AJAR queries to a set of hypergraphs as follows:*

- $C_i^+ = \bigcup_{E \in \mathcal{E}_{C_i}} E$ for all $1 \leq i \leq k$
- $\mathcal{H}_0 = (V(-\alpha), \{F \in \mathcal{E} | F \subseteq V(-\alpha) \cup \{V(-\alpha) \cap C_i^+ | 1 \leq i \leq k\}\})$
- $\mathcal{H}_i^+ = (C_i^+, \mathcal{E}_{C_i} \cup \{V(-\alpha) \cap C_i^+\})$
- $H(\mathcal{H}, \alpha) = \{\mathcal{H}_0\} \cup \bigcup_{1 \leq i \leq k} H(\mathcal{H}_i^+, \alpha_{C_i \setminus C_i^0})$

The hypergraphs in the set $H(\mathcal{H}, \alpha)$ are defined to be the characteristic hypergraphs.

Note that the definition of characteristic hypergraphs depends only on $(\mathcal{H}, \alpha)$, and not on a specific GHD or the instance. Now we state a key result that lets us reduce the problem of searching for an optimal *valid* GHD over $\mathcal{H}$ to that of searching for (not necessarily valid) optimal GHDs over characteristic hypergraphs. Each decomposable GHD corresponds to a GHD over each characteristic hypergraph; conversely, a combination of GHDs for characteristic hypergraphs gives us a decomposable GHD for $\mathcal{H}$. Formally:

THEOREM 34. *For an AJAR query $Q_{\mathcal{H}, \alpha}$, suppose $\mathcal{H}_0, \dots, \mathcal{H}_k$ are the characteristic hypergraphs $H(\mathcal{H}, \alpha)$. Then GHDs $G_0, G_1, \dots, G_k$ of $\mathcal{H}_0, \dots, \mathcal{H}_k$ can be connected to form a decomposable GHD G for $Q_{\mathcal{H}, \alpha}$. Conversely, any decomposable GHD G of $Q_{\mathcal{H}, \alpha}$ can be partitioned into GHDs $G_0, G_1, \dots, G_k$ of the characteristic hypergraphs $\mathcal{H}_0, \dots, \mathcal{H}_k$. Moreover, in both of these cases, $\gamma\text{-width}(G) = \max_i \gamma\text{-width}(G_i)$.*

The proof is provided in the technical report, but it is a straightforward application of definitions.

COROLLARY 35. *Given an optimal GHD for each characteristic hypergraph of an AJAR query $Q_{\mathcal{H}, \alpha}$, we can construct an optimal valid GHD. The width of the optimal valid GHD equals the maximum optimal-GHD-width over its characteristic hypergraphs.*

This reduces the problem of finding the optimal valid GHD to smaller problems of finding optimal GHDs. Then we present several applications of our decomposition.

5.2 Finding optimal valid GHDs

Armed with Corollary 35, we simplify the brute force search algorithm for finding optimal valid GHDs.

THEOREM 36. *Let $Q_{\mathcal{H},\alpha}$ be an AJAR query. The optimal width valid GHD for this query can be found in time $\tilde{O}(|\mathcal{H}|2^{\tilde{O}(\max_{\mathcal{H}' \in H(\mathcal{H},\alpha)}(|\mathcal{H}'|))})$.*

This runtime for finding the optimal valid GHD can be exponentially better than the naive runtime:

EXAMPLE 37. *Consider the star query specified by $\mathcal{H} = (\{A, B_1, \dots, B_n\}, \{\{A, B_i\} \mid 1 \leq i \leq n\})$, $\alpha = (B_1, +), (B_2, +), \dots, (B_n, +)$. A is the only output attribute. Removing A breaks the hypergraph into n components, so there are $n + 1$ characteristic hypergraphs, each of size ≤ 2 . Finding the optimal valid GHD takes time $\tilde{O}(n)$, whereas the standard algorithm takes time exponential in n .*

We can also approximate the GHD [17]:

THEOREM 38 (MARX’S GHD APPROXIMATION). *Let Q be a join query with hypergraph $\mathcal{H}$ and fractional hypertree width w . Then we can find a GHD for Q in time polynomial in $|\mathcal{H}|$, that has width $w' \leq w^3$.*

We can replicate Marx’s result for valid GHDs.

THEOREM 39. *Let $Q_{\mathcal{H},\alpha}$ be an AJAR query, such that its minimum width valid GHD has width w . Then we can find a valid GHD in time polynomial in $|\mathcal{H}|$ that has width $w' \leq w^3$.*

5.3 Tighter Runtime Exponents

Marx [18] introduced the notion of submodular width (sw) that is tighter than fhw , and showed that a join query can be answered in time $\text{IN}^{O(sw)}$. The O in the exponent is because Marx’s algorithm requires expensive preprocessing that takes $\text{IN}^{2 \times sw}$ time. After the pre-processing, the join can be performed in time IN^{sw} . Despite the O in the exponent, this algorithm can be very valuable because there are families of hypergraphs that have unbounded fhw but bounded sw . We can apply Marx’s algorithm to the characteristic hypergraphs, potentially improving our runtime. Marx also showed that joins on a family of hypergraphs are fixed parameter tractable if and only if the submodular width of the hypergraph family is bounded [18]. Moreover, adaptive width [19] (applicable only when relations are expressed as truth tables) is unbounded for a hypergraph family if and only if submodular width is unbounded. Corollary 35 gets us an analogous tractability result for AJAR queries.

THEOREM 40. *We can answer an AJAR query $Q_{\mathcal{H},\alpha}$ in time $O(\text{IN}^{O(\max_{\mathcal{H}' \in H(\mathcal{H},\alpha)}(sw(\mathcal{H}')))} + \text{OUT})$.*

Recent work [14] uses degree information to more tightly bound the output size of a query. The bound in the reference, called the *DBP bound*, has a tighter exponent than the AGM bound, while requiring only linear preprocessing to obtain. The authors also provide algorithms whose runtime matches the DBP bound. We can define DBP-width $dbpw(\mathcal{T}, \mathcal{H})$ such that $\text{IN}^{dbpw(\mathcal{T}, \mathcal{H})}$ is the maximum value of the DBP bound over all bags of GHD T . We then use the improved algorithm in place of GJ in AggroGHDJoin. This lets us get tighter results “for free”, reducing our runtime to IN^{dbpw} instead of IN^{fhw} . Formally:

THEOREM 41. *Given an AJAR query $Q_{\mathcal{H},\alpha}$ and a valid GHD for $\mathcal{H}$, we can answer the query in time $O(\text{IN}^{dbpw(\mathcal{T}, \mathcal{H})} + \text{OUT})$. Equivalently, we can answer the query in time $O(\text{IN}^{\max_{\mathcal{H}' \in H(\mathcal{H},\alpha)} dbpw(\mathcal{T}, \mathcal{H})} + \text{OUT})$.*

5.4 MapReduce and Parallel Processing

The GYM algorithm [3] uses GHDs to efficiently process joins in a MapReduce setting. GYM makes use of the GHD structure to parallelize different parts of the join. Given a GHD of depth d , and width w , with n attributes, GYM can perform a join in a MapReduce setting in $O(d + \log(n))$ rounds at a communication cost of $M^{-1}(\text{IN}^w + \text{OUT})^2$ where M is the memory per processor on the MapReduce cluster. Combining this with the degree-based MapReduce algorithm [14] gives us the following result:

THEOREM 42. *Given an optimal valid GHD $(\mathcal{T}^*, \chi)$ of depth d , and DBP-width $dbpw$, we can answer an AJAR query with Communication Cost equal to $O(M^{-1}(\text{IN}^{dbpw(\mathcal{T}, \mathcal{H})} + \text{OUT})^2)$ in $d + \log(n)$ MapReduce rounds, where n is the number of attributes and M is the available memory per processor.*

A GHD can have depth up to $O(n)$, in which case the algorithm can take a very large number of MapReduce rounds ($O(n)$). To address this, the GYM paper uses the ‘Log-GTA’ algorithm to reduce the depth of any given GHD to $\log(n)$ while at most tripling its width. This lets it process joins in $\log(n)$ MapReduce rounds at a cost of $M^{-1}(\text{IN}^{3w} + \text{OUT})^2$.

Log-GTA involves some shuffling of the attributes in the GHD bags, so naively applying it to a valid GHD could make the GHD invalid (see technical report for an example). But our decomposition lets us apply Log-GTA to the GHD of each characteristic hypergraph, and then stitch the short GHDs together. Our decomposition is recursive in nature; let d' be the maximum recursive depth of the decomposition for a given $Q_{\mathcal{H},\alpha}$. Then the depth of the shortened GHD of each characteristic hypergraph is $O(\log(n))$, and so the depth of the valid GHD obtained by stitching them together is $O(d' \log(n))$. This gives us the result:

THEOREM 43. *If $dbpw$ is the DBP width of an AJAR query, we can answer that query with Communication Cost equal to $O(M^{-1}(\text{IN}^{3 \times dbpw(\mathcal{T}, \mathcal{H})} + \text{OUT})^2)$ in $d' \log(n)$ MapReduce rounds, where n is the number of attributes and M is the available memory per processor.*

d' can vary from $O(1)$ to $O(n)$ depending on the query. The star query from example 37 has $d' = 2$, which lets us process it in $\log(n)$ MapReduce rounds. Any query that only has a single type of aggregation will have $d' = 2$ as well. On the other hand, a query with one relation having n attributes, 1 output attribute, and alternating $\sum$ and $\max$ aggregations, will have $d' = n$, and will be hard to parallelize.

6. PRODUCT AGGREGATIONS

The primary application of queries with multiple aggregations is to establish bounds for the Quantified Conjunctive Query (QCQ) problem [16], and its counting variant, $\#QCQ$. We now introduce a new type of aggregation, called product aggregation, that lets us efficiently handle QCQ queries. We define the AJAR problem for product aggregations, and then extend our algorithm from Section 3 to handle this new type

of AJAR query. We then define a decomposition analogous to that in Section 5. A more detailed version of this section with additional motivation, examples, and proofs can be found in our technical report [13].

6.1 AJAR queries with product aggregates

A *product aggregation* aggregates using the $\otimes$ operator. Throughout the paper, we assumed that an absent tuple effectively has an annotation of 0. Taking this into account, we formally define the product aggregation. Let $B = F \setminus A$:

DEFINITION 44. $\sum_{(A, \otimes)} R_{AB} = \{(t_B, \lambda) \mid \forall t_A \in \mathcal{D}^A, t_B \otimes t_A \in R_{AB} \text{ and } \lambda = \prod_{(t, \lambda_t) \in R_{AB}: \pi_B t = t_B} \lambda_t\}$

We can adjust the definition of aggregation orderings and AJAR queries to allow this new type of aggregation. *QCQ* queries can now be expressed as AJAR queries on the $(\{0, 1\}, \max, \cdot)$ semiring. We assume for this section that $\otimes$ is idempotent, i.e., $a \otimes a = a$ for all a . We describe how to work with non-idempotent products in the technical report.

6.2 Algorithms for product aggregates

For aggregation orderings that have product aggregations, the rules for determining when two orderings are equivalent are somewhat different; product aggregations can be performed *before* a join. We illustrate this with an example:

EXAMPLE 45. In the semiring $(\{0, 1\}, \max, \cdot)$, suppose we have two relations $R(A, B) = \{((0, 0), x), ((0, 1), y)\}$ and $S(B, C) = \{((0, 1), p), ((1, 1), q)\}$. Consider the AJAR query $\sum_{(B, \cdot)} R(A, B) \bowtie S(B, C)$. If we compute the join, we will get two tuples with the annotations $x \cdot p$ and $y \cdot q$, and then aggregating over B will produce a relation with the element $((0, 1), x \cdot p \cdot y \cdot q)$. However, note that $x \cdot p \cdot y \cdot q = (x \cdot y) \cdot (p \cdot q)$, implying that $\sum_{(B, \cdot)} R(A, B) \bowtie S(B, C) = (\sum_{(B, \cdot)} R(A, B)) \bowtie (\sum_{(B, \cdot)} S(B, C))$.

We now describe how to extend each of our results to handle AJAR queries with product aggregations present.

Defining AGHDs and Answering AJAR Queries. Product aggregations change the set of GHDs compatible with a given ordering. In fact, product aggregates allow us to break the running intersection property of GHDs without causing incorrect behavior; a product aggregation attribute can appear in disconnected subtrees without causing incorrect computation. We express this using a simple variant of GHDs, called *aggregating generalized hypertree decompositions* (AGHDs). Informally, AGHDs are GHDs that can violate the running intersection property for attributes that have a product aggregation. We formally define AGHDs by creating new hypergraphs by splitting product aggregation attributes into multiple attributes; in particular, we can define new hypergraphs in which every product aggregation attribute will map to a set of attributes, and AGHDs are equivalent to GHDs over these new hypergraphs.

DEFINITION 46. Given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and aggregation ordering α , let $S = \{a \in \mathcal{V} \mid (a, \otimes) \in \alpha\}$ be the set of attributes with product aggregations. A product partition is a set $\{P_a \mid a \in S\}$ where P_a is a partition of $\{F \in \mathcal{E} \mid a \in F\}$ (the relations that contain a).

We will duplicate each attribute a for each partition of P_a and have the partition specify which edges contain each instance of a . We will use N_a to denote the number of partitions $|P_a|$.

DEFINITION 47. Suppose we are given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$, aggregation ordering α , and product partition P . The product partition hypergraph $\mathcal{H}_P$ is the pair $(\mathcal{V}_P, \mathcal{E}_P)$ such that

- $S = \{a \in \mathcal{V} \mid P_a \in P\}$
- $\mathcal{V}_P = (\bigcup_{a \in S} \{a_1, \dots, a_{N_a}\}) \cup \mathcal{V} \setminus S$
- $p : \mathcal{V} \times \mathcal{E} \rightarrow \mathcal{V}_P$ where $p(a, F) = a$ if $a \notin S$ and $p(a, F) = a_i$ such that F is in the i^{th} partition of P_a otherwise
- $\mathcal{E}_P = \{\{p(a, F) \mid a \in F\} \mid F \in \mathcal{E}\}$

DEFINITION 48. Given a hypergraph $\mathcal{H}$ and aggregation ordering α , an aggregating generalized hypertree decomposition (AGHD) is a triple $(\mathcal{T}, \chi, P)$ such that $(\mathcal{T}, \chi)$ is a GHD of the product partition hypergraph $\mathcal{H}_P$.

For any product aggregation attribute a in the AJAR query, $TOP_{\mathcal{T}}(a)$ for an AGHD $(\mathcal{T}, \chi, P)$ can be defined as the set $\{TOP_{\mathcal{T}}(a_1), TOP_{\mathcal{T}}(a_2), \dots, TOP_{\mathcal{T}}(a_{|P_a|})\}$. Now we can define the notion of compatibility of an AGHD with an ordering.

DEFINITION 49. A AGHD $(\mathcal{T}, \chi, P)$ for an AJAR query $Q_{\mathcal{H}, \alpha}$ is compatible with α if for each attribute pair a, b for which there exists $v_1 \in TOP_{\mathcal{T}}(a)$, $v_2 \in TOP_{\mathcal{T}}(b)$ such that v_1 is an ancestor of v_2 , a must either occur before b in the ordering α or be an output attribute.

We now discuss how to use an AGHD to answer an AJAR query. Given a product partition P , let α^P be a modified aggregation ordering over $\mathcal{V}_P$ that replaces each occurrence of $(a, \otimes)$ in α with $(a_1, \otimes), (a_2, \otimes), \dots, (a_{N_a}, \otimes)$ for each product attribute a . Our definitions imply that if an AGHD $(\mathcal{T}, \chi, P)$ is compatible with the AJAR query $Q_{\mathcal{H}, \alpha}$, $(\mathcal{T}, \chi)$ must be a compatible GHD for the AJAR query $Q_{\mathcal{H}_P, \alpha^P}$. For any instance I over $\mathcal{H}$, let I_P be the corresponding instance over $\mathcal{H}_P$ with the appropriate attributes renamed. To solve $Q_{\mathcal{H}, \alpha}(I)$ with the AGHD $(\mathcal{T}, \chi, P)$, we can run AggroGHDJoin to solve $Q_{\mathcal{H}_P, \alpha^P}(I_P)$ with the GHD $(\mathcal{T}, \chi)$; we will now show that this produces the correct answer.

LEMMA 50. For each database instance I and product partition P , $Q_{\mathcal{H}, \alpha}(I) = Q_{P, \alpha^P}(I_P)$.

This lemma can be proven by repeatedly applying the following lemma.

LEMMA 51. Suppose $R'(A, C_1)$ is a copy of $R(A, C)$ with C renamed to C_1 , and $S'(B, C_2)$ is a copy of $S(B, C)$ with C renamed to C_2 . Then

$$\sum_{(C_1, \otimes)} \sum_{(C_2, \otimes)} R'(A, C_1) \bowtie S'(B, C_2) = \sum_{(C, \otimes)} R(A, C) \bowtie S(B, C)$$

PROOF. Suppose the annotations for the C values in R are $n_1, n_2, \dots, n_k$ and in S are $m_1, m_2, \dots, m_k$ (assume all annotations are present, i.e., absent tuples have a zero-annotation). Then the RHS is $\otimes_{i=1}^k n_i m_i$. The LHS will have $\otimes_{i=1}^k n_i \otimes_{j=1}^k m_j$. The RHS is equal to the LHS because of idempotence of $\otimes$. $\square$

We can thus solve AJAR queries that contain product aggregations with an unchanged algorithm.

Characterizing equivalent orderings. The base case and commutative Lemmas 18 and 21 still hold in our definition of PREC and DNC. However, Lemma 20 does not hold for product aggregations:

EXAMPLE 52. Consider the AJAR query

$$\sum_{(A,+)} \sum_{(B,\max)} \sum_{(C,\cdot)} R(A,C) \bowtie S(B,C).$$

By Lemma 18, we have $\text{PREC}(A,C)$ and $\text{PREC}(B,C)$, and then Lemma 20 implies $\text{PREC}(A,B)$. However, by pushing the aggregations past the join, our query can be rewritten

$$\left(\sum_{(A,+)} \sum_{(C,\cdot)} R(A,C) \right) \bowtie \left(\sum_{(B,\max)} \sum_{(C,\cdot)} S(B,C) \right),$$

which implies that A and B can commute.

LEMMA 53. Given an AJAR query $Q_{\mathcal{H},\alpha}$, suppose $(A, \odot), (B, \odot'), (C, \odot'') \in \alpha$. If $\odot \neq \odot', \odot'' \neq \otimes$, and $\exists E \in \mathcal{E} : \text{PREC}(A,C)$ and $B, C \in E$, then $\text{DNC}(A,B)$.

We adapt Lemma 20 to handle product aggregations correctly by adding the condition that the connecting attribute C does not have a product aggregation.

The proofs to show the completeness and soundness of the resulting partial order $<_{\mathcal{H},\alpha}$ are essentially unchanged, but we do need to adjust the statement of Lemma 24 to add a special case for product aggregations.

LEMMA 54. Suppose we are given a hypergraph $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ and an aggregation ordering α . Suppose $(A, \odot), (B, \odot') \in \alpha$ for differing operators $\odot \neq \odot'$. Then, for any path P in $\mathcal{H}$ between A and B , then there must exist some attribute $C \in P$ such that one of the following is true:

- $C \neq A, B$ and $(C, \otimes) \in \alpha$
- $C <_{\mathcal{H},\alpha} A$ or $C <_{\mathcal{H},\alpha} B$

Decomposing AGHDs. We can apply the ideas from Section 5 to AJAR queries with product aggregates as well. We first extend the definition of compatible GHDs to valid GHDs.

DEFINITION 55. For an AJAR query $Q_{\mathcal{H},\alpha}$, a GHD $(\mathcal{T}, \chi)$ is valid if and only if for each attribute pair a, b for which there exists $v_1 \in \text{TOP}_{\mathcal{T}}(a), v_2 \in \text{TOP}_{\mathcal{T}}(b)$ such that v_1 is an ancestor of v_2 , $b \not<_{\mathcal{H},\alpha} a$.

We can also extend the idea of decomposable GHDs as well. Conceptually the decompositions are very similar: at every stage of the decomposition, we construct a GHD of the output attributes and then we recursively construct decomposable AGHDs on the connected components of the other attributes. The only major difference is that the connected components are computed with the product attributes removed. In particular, decomposable GHDs recurse on the connected components of the hypergraph with the output attributes removed, while decomposable AGHDs recurse on the connected components of the hypergraph with the output

and product aggregation attributes removed. After dividing the attributes into connected components, the product aggregation attributes can be added back to each of the connected components they share an edge with. With this definition, we can easily extend our decomposition and results to AGHDs. We provide the theorem statements here, please see our technical report for details and proofs.

THEOREM 56. All decomposable AGHDs are valid.

THEOREM 57. For every valid AGHD $(\mathcal{T}, \chi, P)$, there exists a decomposable AGHD $(\mathcal{T}', \chi', P')$ such that for all node-monotone functions γ , the γ -width of $(\mathcal{T}', \chi')$ is no larger than the γ -width of $(\mathcal{T}, \chi)$.

We can define characteristic hypergraphs similarly to how we did in Section 5 (again, see the technical report for a formal definition). We have the following result:

THEOREM 58. For an AJAR query $Q_{\mathcal{H},\alpha}$ involving product aggregates, suppose $\mathcal{H}_0, \dots, \mathcal{H}_k$ are the characteristic hypergraphs $H(\mathcal{H}, \alpha)$. Then GHDs $G_0, G_1, \dots, G_k$ of $\mathcal{H}_0, \dots, \mathcal{H}_k$ can be connected to form a decomposable AGHD G for $Q_{\mathcal{H},\alpha}$. Conversely, any decomposable AGHD G of $Q_{\mathcal{H},\alpha}$ can be partitioned into GHDs $G_0, G_1, \dots, G_k$ of the characteristic hypergraphs $\mathcal{H}_0, \dots, \mathcal{H}_k$. Moreover, in both of these cases, $\gamma\text{-width}(G) = \max_i \gamma\text{-width}(G_i)$.

These theorems let us apply all the optimizations from Section 5.2, 5.3, and 5.4 to queries with product aggregates.

7. CONCLUSION

We investigate solutions to and the structure of AJAR queries: aggregate-join queries with multiple aggregators over annotated relations. We start by providing a simple algorithm based on a variant of GHDJoin that operates with a limited set of GHDs. We proceed to investigate the class of equivalent orderings to expand the set of GHDs that we can give our algorithm. To that end, we develop a partial ordering that fully characterizes equivalent orderings. We then develop a characterization of the corresponding valid GHDs, describing how they can be decomposed into ordinary, unrestricted GHDs. This reduction connects us to a trove of parallel work on GHDs. We finish by extending our work to handle product aggregations.

Acknowledgements. We thank Atri Rudra for invaluable insights and feedback developing our approach. CR gratefully acknowledges the support of the Defense Advanced Research Projects Agency (DARPA) XDATA Program under No. FA8750-12-2-0335, DARPA's SIMPLEX Program, DARPA's MEMEX program under No. FA8750-14-2-0240, the National Science Foundation (NSF) under CAREER Award No. IIS-1353606, Award No. CCF-1356918 and EarthCube Award under No. ACI-1343760, the Office of Naval Research (ONR) under awards No. N000141210041 and No. N000141310129, the Sloan Research Fellowship, the Moore Foundation Data Driven Investigator award, and gifts from American Family Insurance, Intel, Lightspeed Ventures, and Toshiba.

8. REFERENCES

- [1] Christopher R. Aberger, Susan Tu, Kunle Olukotun, and Christopher Ré. Emptyheaded: A relational engine for graph processing. *SIGMOD (to appear)*, 2016.
- [2] Serge Abiteboul, Richard Hull, and Victor Vianu, editors. *Foundations of Databases: The Logical Level*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 1st edition, 1995.
- [3] Foto N. Afrati, Manas Joglekar, Christopher Ré, Semih Salihoglu, and Jeffrey D. Ullman. GYM: A multiround join algorithm in mapreduce. *CoRR*, abs/1410.4156, 2014.
- [4] Srinivas M. Aji and Robert J. McEliece. The generalized distributive law. *IEEE Transactions on Information Theory*, 46(2):325–343, 2000.
- [5] Albert Atserias, Martin Grohe, and Dániel Marx. Size bounds and query plans for relational joins. FOCS ’08, pages 739–748, Washington, DC, USA, 2008. IEEE Computer Society.
- [6] Nurzhan Bakibayev, Tomáš Kociský, Dan Olteanu, and Jakub Zavodny. Aggregation and ordering in factorised databases. *PVLDB*, 6(14):1990–2001, 2013.
- [7] Hubie Chen and Víctor Dalmau. Decomposing quantified conjunctive (or disjunctive) formulas. In *Proceedings of the 27th Annual IEEE Symposium on Logic in Computer Science, LICS 2012, Dubrovnik, Croatia, June 25-28, 2012*, pages 205–214, 2012.
- [8] B. Eckmann and P. J. Hilton. Group-like structures in general categories i multiplications and comultiplications. *Mathematische Annalen*, 145(3):227–255.
- [9] G. Gottlob, M. Grohe, M. Nysret, M. Samer, and F. Scarcello. Hypertree Decompositions: Structure, Algorithms, and Applications. In *WG*, 2005.
- [10] Georg Gottlob, Nicola Leone, and Francesco Scarcello. Hypertree decompositions and tractable queries. In *PODS 1999*, pages 21–32, 1999.
- [11] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. PODS ’07, pages 31–40, New York, NY, USA, 2007. ACM.
- [12] Martin Grohe and Dániel Marx. Constraint solving via fractional edge covers. *ACM Trans. Algorithms*, 11(1):4:1–4:20, August 2014.
- [13] Manas Joglekar, Rohan Puttagunta, and Christopher Ré. Ajar: Aggregations and joins over annotated relations. *CoRR*, abs/1508.07532, 2015.
- [14] Manas R. Joglekar and Christopher Ré. It’s all a matter of degree: Using degree information to optimize multiway joins. In *ICDT (to appear)*, 2016.
- [15] Kaleb Kask, Rina Dechter, Javier Larrosa, and Avi Dechter. Unifying tree decompositions for reasoning in graphical models. *Artif. Intell.*, 166:165–193, 2005.
- [16] Mahmoud Abo Khamis, Hung Q. Ngo, and Atri Rudra. FAQ: questions asked frequently. *CoRR*, abs/1504.04044, 2015.
- [17] Dániel Marx. Approximating fractional hypertree width. *ACM Trans. Algorithms*, 6, 2010.
- [18] Dániel Marx. Tractable hypergraph properties for constraint satisfaction and conjunctive queries. In *Proceedings of the Forty-second ACM Symposium on Theory of Computing, STOC ’10*, pages 735–744, New York, NY, USA, 2010. ACM.
- [19] Daniel Marx. Tractable structures for constraint satisfaction with truth tables. *Theory of Computing Systems*, 48, 2011.
- [20] Hung Q. Ngo, Ely Porat, Christopher Ré, and Atri Rudra. Worst-case optimal join algorithms: [extended abstract]. In *PODS 2012*, pages 37–48, 2012.
- [21] Hung Q. Ngo, Christopher Ré, and Atri Rudra. Skew strikes back: new developments in the theory of join algorithms. *SIGMOD Record*, 42(4):5–16, 2013.
- [22] Dan Olteanu and Jakub Zavodný. Size bounds for factorised representations of query results. *ACM Trans. Database Syst.*, 40(1):2, 2015.
- [23] Adam Perelman and Christopher Ré. Duncenap: Compiling worst-case optimal query plans. In *SIGMOD 2015*, pages 2075–2076, 2015.
- [24] Christopher Ré and Dan Suciu. The trichotomy of HAVING queries on a probabilistic database. *VLDB J.*, 18(5):1091–1116, 2009.
- [25] Neil Robertson and P.D Seymour. Graph minors. iii. planar tree-width. *Journal of Combinatorial Theory, Series B*, 36(1):49 – 64, 1984.
- [26] Charles A. Sutton and Andrew McCallum. An introduction to conditional random fields. *Foundations and Trends in Machine Learning*, 4(4):267–373, 2012.
- [27] Susan Tu and Christopher Ré. Duncenap: Query plans using generalized hypertree decompositions. In *SIGMOD 2015*, pages 2077–2078, 2015.
- [28] Todd L. Veldhuizen. Triejoin: A simple, worst-case optimal join algorithm. In *ICDT 2014*, pages 96–106, 2014.
- [29] M. Yannakakis. Algorithms for Acyclic Database Schemes. In *VLDB*, 1981.

APPENDIX

A. ALGORITHMS

We provide pseudo-code of AggroGHDJoin in Section 4. The last line of this algorithm calls AggroYannakakis, which we now outline in Algorithm 2. AggroYannakakis and the classic Yannakakis algorithm for project-join queries are essentially the same; AggroYannakakis is slightly modified to handle aggregations instead of simple projections.

In the classic analysis of Yannakakis for join queries (without projection), the runtime of the semi-join portion is bounded by $O(\text{IN})$ and the bottom-up join is bounded by $O(\text{OUT})$. Adding projection (or aggregation) leaves the semi-join portion unchanged, but the projection reduces the size of the output, thereby making the OUT bound harder to achieve. In particular, during the bottom-up join, we may compute an intermediate relation whose attributes are not a subset of the output attributes, meaning that its size may not be bounded by OUT. These potentially large intermediate relations are the underlying cause for the traditional $\text{IN} \cdot \text{OUT}$ runtime.

However, using intuition discovered in reference [6], if we require the output attributes to appear above non-output attributes, we can preserve the $\text{IN} + \text{OUT}$ runtime.

THEOREM 59. *Suppose we have a GHD such that for any output attribute A and non-output attribute B , $\text{TOP}_T(B)$ is not an ancestor of $\text{TOP}_T(A)$. AggroGHDJoin runs in $\tilde{O}(\text{IN}^{\text{flow}(\mathcal{T}, \mathcal{H}, I)} + \text{OUT})$ given this GHD.*

Algorithm 2 AggroYannakakis($\mathcal{T} = (\mathcal{V}, \mathcal{E}), \alpha, \{R_F | F \in \mathcal{V}\}$)

Input: Join tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$, Aggregation order α , Relations R_F for each $F \in \mathcal{V}$

```

for all  $F \in \mathcal{V}$  in some bottom-up order do    ▷ Semi-join
  reduction up
     $P \leftarrow$  parent of  $F$ 
     $R_P \leftarrow R_P \times R_F$ 
end for
for all  $F \in \mathcal{V}$  in some top-down order do    ▷ Semi-join
  reduction down
     $P \leftarrow$  parent of  $F$ 
     $R_F \leftarrow R_F \times R_P$ 
end for
while  $F \in \mathcal{V}$  in some bottom-up order do ▷ Aggregation
   $\beta \leftarrow \alpha \cap \{a \in \mathcal{V} | TOP_{\mathcal{T}}(a) = F\}$ 
   $R_F \leftarrow \Sigma_{\beta} R_F$ 
  if  $F$  is not the root then
     $P \leftarrow$  parent of  $F$ 
     $R_P \leftarrow R_P \bowtie R_F$     ▷ Compute the join
  end if
end while
return  $R_R$  for the root  $R$ 

```

PROOF. GJ on each bag still runs in $\tilde{O}(\text{IN}^{fhw(\mathcal{T}, \mathcal{H}, I)})$. We need to prove the Yannakakis portion runs in $O(\text{IN} + \text{OUT})$ after running GJ .

The semijoin portion runs in $O(\text{IN})$ as in the original Yannakakis algorithm. In the join phase, we have two types of joins. In the first type, $F \setminus \beta \subseteq P$. This implies the join output is a subset of R_P (with different annotations). So the total runtime of this type of join is $O(\text{IN})$. For the second type, $F \setminus \beta \not\subseteq P$. This means some attribute in $(F \setminus \beta) \setminus P$ must be an output attribute, and all attributes in P must be output attributes as well (as their TOP value is an ancestor of F). So the result of our join must be a subset of the output table; the total runtime of this type of join is $O(\text{OUT})$. Thus the total runtime of the algorithm is $O(\text{IN} + \text{OUT})$. $\square$

Note that while AggroGHDJoin runs in $\tilde{O}(\text{IN}^{fhw(\mathcal{T}, \mathcal{H}, I)} + \text{OUT})$ time on the GHDs above, it may not necessarily produce the right output unless the GHD satisfies additional conditions, to ensure that aggregations are done in the proper order. In particular, recall our definition a GHD $(\mathcal{T}, \chi)$ is compatible with α if for all attribute pairs A, B , $TOP_{\mathcal{T}}(A)$ being an ancestor of $TOP_{\mathcal{T}}(B)$ implies that either A is an output variable or A occurs before B in α .

THEOREM 60. *If a GHD $(\mathcal{T}, \chi)$ is compatible with α , then AggroGHDJoin given $(\mathcal{T}, \chi)$ correctly computes $Q_{\mathcal{H}, \alpha}$.*

PROOF. We first show that AggroYannakakis works as expected. We note that the semi-join reduction does not change the output; it only quickens the process. We only consider the bottom-up join. For each node t in the join tree, let $R(t)$ be the relation associated with that node before this loop, i.e., after the semi-join portion. Let $R'(t)$ be the final relation associated with node t when we are processing node t , i.e., after the bottom up join with t 's descendants is done and after the aggregation in t . Let $\mathcal{T}_t$ be the subtree that includes t and all of its descendants. Let $s(t)$ be the attributes aggregated at node t , i.e., $\alpha \cap \{a \in \mathcal{V} | TOP_{\mathcal{T}}(a) = t\}$, and let $s(\mathcal{T}_t) = \cup_{t' \in \mathcal{T}_t} s(t')$. For each non-leaf node t , let $c(t)$ be the set of t 's children.

For each node t , we claim $R'(t) = \sum_{\alpha_{s(\mathcal{T}_t)}} \bowtie_{t' \in \mathcal{T}_t} R(t')$. Proof by induction on the tree. For each leaf l , $R'(l) = \sum_{\alpha_{s(l)}} R(l)$ by definition.

For a non-leaf node t ,

$$\begin{aligned}
 R'(t) &= \sum_{\alpha_{s(t)}} R(t) \bowtie (\bowtie_{t_c \in c(t)} R'(t_c)) \\
 &= \sum_{\alpha_{s(t)}} R(t) \bowtie \left(\bowtie_{t_c \in c(t)} \sum_{\alpha_{s(\mathcal{T}_{t_c})}} \bowtie_{t' \in \mathcal{T}_{t_c}} R(t') \right) \\
 &= \sum_{\alpha_{s(\mathcal{T}_t)}} \bowtie_{t' \in \mathcal{T}_t} R(t')
 \end{aligned}$$

The second step is due to the inductive hypothesis. The final step is simply “pulling out” the aggregations from the sub-orderings one at a time; we can arbitrarily interleave the aggregation orders $\alpha_{s(\mathcal{T}_{t_c})}$. We can simply interleave them to match $\alpha_{\cup_{t_c \in c(t)} s(\mathcal{T}_{t_c})}$. Since the original GHD is compatible with α , we know the aggregations $\alpha_{s(t)}$ precede $\alpha_{s(\mathcal{T}_{t_c})}$ in α , implying that $\sum_{\alpha_{s(\mathcal{T}_t)}} \sum_{\alpha_{\cup_{t_c \in c(t)} s(\mathcal{T}_{t_c})}} = \sum_{\alpha_{s(\mathcal{T}_t)}}$. Our output is $R'(t_r)$ where t_r is the root node, which is $\sum_{\alpha} \bowtie_{t \in \mathcal{T}} R(t)$ as desired.

Since AggroYannakakis works as expected, we simply need to ensure that the bags are computed appropriately. Note the GHD ensures for every relation R_F , there is a node t such that $F \subseteq \chi(t)$. This means that no tuple is lost; computing AggroYannakakis on the bags will compute the correct tuples. To ensure it computes the correct annotations, we need to ensure every annotation appears in the bags at most once; our algorithm places the annotation of a relation R_F in the top-most node that contains all of the attributes R_F . $\square$

B. COMPARISON WITH RELATED WORK

We compare our results with prior work. We follow the structure of our paper, comparing the runtimes of AggroGHDJoin, the ordering equivalence characterization, GHD decomposition results, and finally product aggregation results.

B.1 Section 3

In Section 3, we define a simple approach to solving AJAR queries, namely running AggroGHDJoin over a minimum valid GHD. Our runtime is given by $O(\text{IN}^{w^*} + \text{OUT})$, where w^* is the width of the optimal GHD (Theorem 10). In contrast, the runtime of the FAQ's InsideOut algorithm (shown in Algorithm 3) is $O(\text{IN}^{faqw})$, where $faqw$ is called the FAQ-width of the query. We show that $\tilde{O}(\text{IN}^{w^*} + \text{OUT}) \leq \tilde{O}(\text{IN}^{faqw})$. We note that the $faqw$ exponent is actually the optimum value of $faqw(\sigma)$ over the equivalent orderings σ FAQ considers (we discuss the space of orderings it considers in the next subsection). Our approach will recognize σ as being equivalent, and will search for the best compatible GHD for σ . We will show that there exists a compatible AGHD $(\mathcal{T}, \chi, P)$ for every equivalent ordering σ such that $fhw(\mathcal{T}, \mathcal{H}, I) = faqw(\sigma)$ (as Example 62 shows, the compatible AGHD $\mathcal{T}$ we construct may not be the optimal compatible GHD).

We start by briefly summarizing FAQ's InsideOut algorithm, with the pseudo-code (written in the notation of this paper) given in Algorithm 3. Let σ be the ordering used for aggregation. Let n denote the total number of attributes

$|\mathcal{V}_H|$ and f denote the number of output attributes (thus $|\sigma| = n - f$). For notational convenience, we will be using $\sigma[i]$ to denote both the attribute and the operator that make up the i^{th} operator-attribute pair in the ordering.

Algorithm 3 InsideOut($\mathcal{H} = (\mathcal{V}_H, \mathcal{E}_H)$, σ , $\{R_F | F \in \mathcal{E}_H\}$)

Input: Hypergraph $\mathcal{H} = (\mathcal{V}_H, \mathcal{E}_H)$, Aggregation ordering σ , Relations R_F for each $F \in \mathcal{E}_H$

```

 $E_n \leftarrow \{R_F \mid F \in \mathcal{E}_H\}$ 
for ( $k = n; k > f; k--$ ) do
   $\delta(k) \leftarrow \{R_F \in E_k \mid \sigma[k-f] \in F\}$ 
  if  $\sigma[k-f]$  is not a product aggregation then
     $U_k \leftarrow \bowtie_{R \in \delta(k)} R$ 
     $E_{k-1} \leftarrow (E_k \setminus \delta(k)) \cup \{\sum_{\sigma[k-f]} U_k\}$ 
  else
     $E_{k-1} \leftarrow (E_k \setminus \delta(k)) \cup \{\sum_{\sigma[k-f]} R \mid R \in \delta(k)\}$ 
  end if
end for
return  $\bowtie_{R \in E_f} R$ 

```

FAQ relies on a worst-case optimal algorithm to compute each of the joins, implying that in the $\tilde{O}(\text{IN}^{faqw})$ runtime guarantee, $faqw$ is defined as the maximum AGM bound placed on each of the computed joins. Define $p_H^* : 2^{\mathcal{V}_H} \rightarrow \mathcal{R}$ to be a function that maps a subset of the attributes to the AGM bound on the subset, i.e., the optimal value of the canonical linear program. Then $faqw = \max(\max_k p_H^*(U_k), p_H^*(V(-\sigma)))$ [16].

We will build up the compatible AGHD $(\mathcal{T}, \chi, P)$ in rounds corresponding to each of the k values of *InsideOut*. We first describe how to construct $(\mathcal{T}, \chi)$, and later describe how to obtain P . At the start of round corresponding to a particular k , we will have a forest of AGHDs, each of which will have a root mapped (by χ) to the attribute sets of E_k , and at the end of each round, the forest's roots will be mapped to the relations of E_{k-1} .

For an attribute set F , let $t(F)$ represent the node such that $\chi(t(F)) = F$. We start by creating the $|\mathcal{E}_H|$ nodes $\{t(F) \mid F \in E_n\}$, which are simply nodes mapped to the input relations. Then for each k from n to $f + 1$, let T represent the set of nodes $\{t(F) \mid F \in \delta(n)\}$; these are the nodes that will be processed, i.e., the nodes for whom we will create parents. If $\sigma[k-f]$ is not a product aggregation, we create a node $t(U_k)$ and set $\text{parent}(t) = t(U_k)$ for all $t \in T$. We then create a node $t(U_k \setminus \{\sigma[k-f]\})$ and set it to be $\text{parent}(t(U_k))$. Note that this process has transformed the set of the forest's roots by removing T and adding $t(U_k \setminus \{\sigma[k-f]\})$, mirroring the transformation between E_k and E_{k-1} . If $\sigma[k-f]$ is a product aggregation, then for each $F \in \delta(n)$, we create a node $t(F \setminus \{\sigma[k-f]\})$ and set it to be $\text{parent}(t(F))$; in this case as well the set of the forest's roots match E_{k-1} .

At the end of this process, we will have a forest of AGHDs whose roots map to the relations in E_f . To conclude our construction, we simply construct the node $t(V(-\sigma))$ and set it to be $\text{parent}(t(F))$ for all $F \in E_f$. If there are no product aggregations, then $(\mathcal{T}, \chi)$ forms a GHD.

$(\mathcal{T}, \chi)$ satisfies the running intersection property for all non-product attributes, but a product attribute a can be present in multiple disconnected parts of $\mathcal{T}$. We now describe a product partition P such that $(\mathcal{T}, \chi, P)$ forms an AGHD for the ordering σ . Let P_a denote the number of distinct

connected components of $\mathcal{T}$ in which a is present. Then we create P_a copies of a ($a_1, a_2, \dots, a_{|P_a|}$), and assign a copy to each component in some order. For each $F \in \mathcal{E}_H$ that contains a , if the component that $t(F)$ belongs to is assigned a_i , then P assigns a_i to F . Then $(\mathcal{T}, \chi, P)$ is an AGHD for σ .

The AGHD $(\mathcal{T}, \chi, P)$ as described is trivially compatible with σ since we construct $\text{parent}(\text{TOP}_{\mathcal{T}}(\sigma[k-f]))$ explicitly in round k ; this ensures that $\text{TOP}_{\mathcal{T}}(\sigma[i])$ cannot be an ancestor of $\text{TOP}_{\mathcal{T}}(\sigma[j])$ if $i > j$.

LEMMA 61. *Define p_H^* to be a function that maps a set of attributes to the AGM bound on the set (the optimal value of the canonical linear program). The AGHD $(\mathcal{T}, \chi, P)$ constructed as described satisfies*

$$\text{fhw}(\mathcal{T}, \mathcal{H}, I) = \max(p_H^*(V(-\alpha)), \max_k p_H^*(U_k)) = \text{faqw}(\sigma).$$

PROOF. The nodes in our tree that do not map to $V(-\alpha)$ or the U_k either map to an input relation or to a relation created by aggregating an attribute from a single child node. In the former case, p_H^* would evaluate to 1, so we can ignore them in our maximum. In the latter case, the attributes are a strict subset of its child's attributes, implying we can ignore them too. As such, the fractional hypertree width is simply the maximum fractional cover over $V(-\alpha)$ and the U_k . This shows the first part of the equality. The second part of the equality is the definition of $faqw$ [16]. $\square$

Our runtime comparison for regular as well as product aggregations, follows as a simple corollary. We now show an example where the runtime of InsideOut is much higher than that of AggroGHDJoin, primarily because InsideOut is not output-sensitive.

EXAMPLE 62. *Let n be an even number, and consider an AJAR query $Q_{H,\alpha}$ where $\mathcal{H} = (\{A_i \mid 1 \leq i \leq n\}, \{\{A_i, A_{i+1}\} \mid 1 \leq i \leq n-1\} \cup \{\{A_n, A_1\}\})$, and α is empty, i.e., the query is just a join. Also let each attribute take values $1, 2, 3, \dots, 2 \times \lfloor \sqrt{N} \rfloor$. Suppose each relation $\{A_i, A_{i+1}\}$ for $1 \leq i \leq n-1$ connects values of the same parity, while relation $\{A_n, A_1\}$ connects values of opposite parities. Thus each relation has size N , and $\text{IN} = O(N)$ (n is a constant), and the join output is empty. There is a GHD with bags $\{A_1, A_2, A_3\}, \{A_1, A_3, A_4\}, \dots, \{A_1, A_{n-1}, A_n\}$ that is compatible with the empty ordering. The fhw of this GHD is 2, so we have $w^* = 2$. Thus the runtime of our algorithm will be $\tilde{O}(\text{IN}^2)$. InsideOut will compute an intermediate output consisting of the join of $n-1$ of the relations, which has size $N^{(n-1)/2}$, so InsideOut's runtime will be at least $\tilde{O}(\text{IN}^{(n-1)/2})$.*

The FAQ paper does discuss, at a very high-level and without proofs, changes to InsideOut that will allow its runtime to be output-sensitive [16, Section 10.2]. It's most general and useful change involves building a GHD for the output variables and running a message passing algorithm between the bags, which exactly describes GHDJoin. Implementing this change would make InsideOut completely equivalent to AggroGHDJoin. We note that the FAQ paper frames these changes as decisions in how to *represent* the output, whereas we present the optimization in an algorithmic context, independent of any other storage optimizations.

B.2 Section 4

In Section 4, we define a partial order $<_{\mathcal{H},\alpha}$ that exactly characterizes the constraints an aggregation ordering must satisfy to be equivalent to a given ordering α . Our partial ordering is complete. The FAQ paper also defines its own partial ordering, which we denote $<_{FAQ}$, and it only considers orderings that are linear extensions of $<_{FAQ}$. However, $<_{FAQ}$ is not complete, as shown in the following example:

EXAMPLE 63. Consider the AJAR query given by $\sum_A \max_B \sum_C R(A, B)S(A, C)$. By our characterization, $A <_{\mathcal{H},\alpha} B$ is the only constraint, giving rise to 3 different valid orderings. The FAQ characterization, however, has two constraints: $A <_{FAQ} B$ and $C <_{FAQ} B$, which only allows for 2 different valid orderings. Note that FAQ constraints preclude the original ordering ABC .

B.3 Section 5

In Section 5, we define a decomposition that relates the width of a valid GHD to the widths of a series of ordinary GHDs. Variable orderings (as used by FAQ) are not as readily suited as GHDs are for decompositions. FAQ does have its own version of a decomposition, but the difficulties that arise when using variable orderings are exemplified in the way FAQ switches between GHDs and variable orderings in proofs [16]. In addition, the FAQ decomposition incurs some overhead costs when combining the sub-orderings to build the overall ordering, precluding a result like Corollary 35 that is required for our extensions. We briefly describe this overhead cost, and then provide an example of the runtime difference it can cause.

Output Addition. The FAQ paper uses a decomposition of the problem that is not width-preserving. It removes the set of output attributes $V(-\alpha)$ and decomposes the rest of the hypergraph into smaller hypergraphs. Then it constructs a regular Variable-Ordering/GHD for each hypergraph. Then it adds all output attributes $V(-\alpha)$ into each bag of each of the GHDs, and then stitches the GHDs together. This *output addition* to the bags of the GHDs leads to a potentially $2\times$ increase in width compared to our method which stitches the GHDs together without changing their width. As a result, FAQ’s decomposition incurs higher runtime costs in each application of the decomposition, as we see in the next three subsections.

EXAMPLE 64. Consider the query $\sum_B \sum_C \sum_D R(A, B)S(B, C)T(C, D)U(D, A)$. Suppose $|A| = \sqrt{N}$, $|B| = 2 = |D|$, $|C| = N$, and all of the pairwise relations are constructed as complete cross products of the attributes’ values. Our decomposition will result in the chain GHD $A-ABD-BCD$, while the FAQ decomposition will result in the GHD $A-ABC-ACD$. The runtimes of both FAQ and GHDJoin using the former GHD is $\tilde{O}(N)$, whereas the runtimes using the latter GHD are $\tilde{O}(N^{3/2})$. As such, the FAQ decomposition will perform asymptotically worse than our decomposition.

More generally, consider a query Q_n with relations $R_i(A_i, B)$, $S_i(B, C_i)$, $T_i(C_i, D)$, $U_i(D, A_i)$ for $1 \leq i \leq n$. Like before, all $|A_i|$ ’s are $\sqrt{N}$, all $|C_i|$ ’s are N , and $|B| = |D| = 2$. And

the aggregation ordering only has the $+$ operator, on B , D and all C_i ’s. Our decomposition gives the chain $A_1 \dots A_n - A_1 \dots A_n BD - C_1 \dots C_n BD$. This results in a runtime of $\tilde{O}(N^n)$. FAQ’s decomposition gives $A_1 \dots A_n BC_1 \dots C_n - A_1 \dots A_n DC_1 \dots C_n$. This decomposition, and its corresponding ordering, give a runtime of $\tilde{O}(N^{3n/2})$. Thus the difference between runtime exponents caused by FAQ’s decomposition and our decomposition can be arbitrarily high.

B.4 Section 5.2

Section 5.2 shows how we can construct an approximately optimal valid GHD using Marx’s approximation. FAQ [16]’s decomposition lets it apply Marx’s approximation as well, but gives a weaker width guarantee of $faqw^3 + faqw$ [16, Theorem 9.49]. The extra $+faqw$ factor is due to output addition.

EXAMPLE 65. Consider a query with output attribute A

$$\sum_{B,+} \sum_{C,+} (R(A, B) \bowtie S(B, C)).$$

The optimal valid GHD for this query has bags $\{A, B\}$ and $\{B, C\}$, and thus has fhw 1. The $faqw$ is also 1. If we apply our decomposition, we get a GHD with bags $\{A\}$, $\{A, B\}$, $\{B, C\}$ which still has fhw 1. FAQ’s decomposition on the reduced hypergraph (with output attribute A removed) has one bag $\{B, C\}$. Adding A to it gives a single bag $\{A, B, C\}$ resulting in a fhw of 2. More generally, consider query Q_n with $\alpha = ((B_1, +), (B_2, +), \dots, (B_n, +))$ and relations $T(A_1, B_1)$ and also $R_{i,j}(A_i, A_j)$, $S_{i,j}(B_i, B_j)$ for $i, j \in \{1, 2, \dots, n\}$. Our decomposition gives a GHD with bags $\{A_1, A_2, \dots, A_n\}$, $\{A_1, B_1\}$, $\{B_1, B_2, \dots, B_n\}$, which has fhw $n/2$. FAQ’s decomposition has a single bag and fhw equal to n .

Because of output addition, FAQ incurs an extra cost in our extensions involving DBP-width (Section 5.3) and MapReduce (Section 5.4) as well. We provide details and examples in the technical report.

B.5 Section 6

The runtime of InsideOut on a query involving idempotent product aggregations is given by $\tilde{O}(\text{IN}^{faqw})$, where the $faqw$ depends on the ordering, and the presence of product aggregations. Our algorithm for handling product aggregations recovers the runtime of FAQ. Formally,

THEOREM 66. For any AJAR query involving idempotent product aggregations, $\text{IN}^{w^*} + \text{OUT} \leq 2 \cdot \text{IN}^{faqw}$.

The proof is in the technical report. By applying ideas from the FAQ paper to our setting, we can also recover the FAQ runtime on $\#QCQ$. Our order equivalence characterization for product aggregations is both sound and complete; FAQ’s characterization algorithm is sound but not complete. We also have a width-preserving decomposition for queries with product aggregates. This lets us get tighter runtime exponents in terms of submodular and DBP-widths (Theorems 40, 41) and efficient MapReduce Algorithms (Theorems 42, 43).